# FacePass Android Developer Guide V3.13.x

---

This document is the FacePass Android client SDK documentation from Megvii Technology, designed for users. It provides a comprehensive introduction to the client SDK's classes, member variables, member functions, and practical usage tips. For actual integration, please use the accompanying sample Demo project.

The document directory is as follows:

# SDK direction for use

## SDK brief introduction

FacePassAndroidSDK is a library for Android app development.

To develop Android applications with SDK, you need to add SDK package FacePassAndroidSDK-release.aar to the project, and put the detector_rect.arm.C.bin,feat2.arm.C.v1.0.1core.bin and other bin model files required by the algorithm into the assets directory of the project.

The Android SDK's function is to detect and perform facial recognition operations, including liveness detection and face database retrieval.

Common workflow:

1. SDK initialization

2. Face database

3. Face detection

4. Face recognition

# SDK core classes and functions

The FacePass SDK interface utilizes multiple internally defined structures, as detailed below.

## FacePassConfig

### Summary

The configuration class for the FacePassSDK handler.

Main scenarios:

1. Initialize all configurations and complete the initialization of the core engine FacePassHandler as the primary parameter.

2.FacePassHandler provides get/set functions for users to query and modify configurations.

The public parameter of FacePassConfig is divided into four main categories:

1. Threshold and switch classes: Provide thresholds for facial recognition and liveness detection to determine whether to approve.

2. Quality assessment: For captured facial frames, evaluate quality parameters including angle, blurriness, and lighting to select compliant faces for recognition.

3. Business parameters: Provides parameters such as retry attempts for failed recognition, camera rotation angle, and file storage path.

4. Model files: These are the models required by the algorithm for configuration and initialization, with validation performed based on the mode type. The snapshot mode loads 3 model files, while the offline mode requires 6. The logic for actual thresholds and switches is primarily documented in the return values of the FacePassSDK handler's feedFrame interface.

The actual quality judgment parameters and other parameters will be explained in detail in the following table.

The actual model file is explained in the data structure of FacePassModel.

| Variable Classi-fication | Classification declaration | Variable name | Type of variable | Variable declaration |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Threshold and Switch | A complete facial recognition process involves two algorithms:<br>• Face recognition (Recognition): The system retrieves the highest-scoring candidate from the database and checks if it exceeds the threshold. If the score is above the threshold, the candidate is approved; otherwise, it is rejected.<br>• Liveness detection: This feature evaluates facial recognition by comparing the score against a predefined threshold. If the score exceeds the threshold, the system confirms the subject is a real person; otherwise, it flags the detection as an attack.<br>Access control systems will only permit entry for faces that meet both Search and Liveness criteria.<br>The higher the search threshold, the stricter the requirements, making both correct and false positives harder to achieve—a trade-off. Additionally, different database sizes yield varying recommended thresholds. The same applies to Liveness thresholds: stricter criteria make it harder to distinguish true from false positives, again reflecting a balance.<br>We will provide a default threshold, but customers can also adjust it based on their actual needs.<br>The live body provides a switch to disable live body detection for improved performance. | searchThreshold | float | Recognition threshold 。<br>Distribution range [0,100], float type, with no digit limit.<br>The extreme values 0 and 100 are supported, but we strongly advise against using these values in practice, as it may cause the effect to become unavailable.<br>For specific version thresholds, refer to the corresponding Release note.<br>*Note: The above pass/fail rate is based on normal scenarios with qualified database quality. The actual recognition rate depends on factors such as database quality, on-site quality judgment thresholds, and camera imaging, and is subject to user experience.<br>When the threshold is raised, the pass rate within the base database decreases (a negative effect), while the misidentification rates both within and outside the database also decline (positive effects). Conversely, the missed recognition rate within the database increases (a negative effect). In practice, achieving an "optimal" threshold adjustment is a delicate process that generally doesn't require excessive rigor. For databases with fewer than 10,000 entries, the current recommended threshold precision usually suffices. If any rate becomes excessively abnormal, we can assist with data analysis and provide on-site threshold recommendations. |
| | | livenessThreshold | float | Living threshold.<br>Distribution range [0,100], float type, with no digit limit.<br>The extreme values 0 and 100 are supported, but we strongly advise against using these values in practice, as it may cause the effect to become unavailable.<br>For specific version thresholds, refer to the corresponding Release note.<br>*Note: The actual pass rate depends on the field environment, attack type, etc.<br>This threshold is designed for access control scenarios and ensures the default pass rate (smooth passage for personnel in the base database). If different scenarios have different requirements (such as an extremely high attack prevention rate), you can adjust the threshold to meet the needs.<br>The method is to raise the threshold, the pass rate of real person and attack will be reduced, and the required pass rate can be reached; the opposite idea is the same. |
| | | livenessEnabled | bool | Living switch.<br>True or false, choose one.<br>To use a live model, set it to true when initializing FacePassHandler and pass the model. Alternatively, pass only the model during initialization, then enable it through the setConfig interface later.<br>Liveness detection consumes computational resources. In scenarios where it is unnecessary (e.g., manned environments), disabling the liveness detection feature can enhance the overall performance of the algorithm system. |
| | | rgbIrLivenessEnabled | bool | Infrared live switch is off by default.<br>True or false, choose one.<br>To use the infrared live model, set it to true when initializing FacePassHandler and pass the model. Alternatively, pass only the model during initialization, then enable it through the setConfig interface later.<br>A RGB+IR binocular camera is required.<br>Liveness detection consumes computational resources. In scenarios where it is unnecessary (e.g., manned environments), disabling the liveness detection feature can enhance the overall performance of the algorithm system.<br>Infrared and optical liveness detection are two independent functions that operate independently. The SDK only utilizes one liveness detection algorithm, prioritizing infrared detection. When users need to switch between monocular and dual-camera liveness detection, both models must be initialized simultaneously during handle creation. Subsequently, the following call enables seamless hot-swapping between monocular and dual-camera liveness detection, with the model switching automatically during the next recognition process.<br>//Switch to monocular live view<br>`FacePassConfig faceConfig =mFacePassHandler.getConfig();`<br>`faceConfig. livenessEnabled = true;`<br>`faceConfig. rgbIrLivenessEnabled= false;`<br>`mFacePassHandler.setConfig();`<br>//Switch to dual-camera live detection<br>`FacePassConfig faceConfig =mFacePassHandler.getConfig();`<br>`faceConfig. livenessEnabled = false;`<br>`faceConfig. rgbIrLivenessEnabled= true;`<br>`mFacePassHandler.setConfig();` |
| | | smileEnabled | bool | The Smile Model Enable Switch is off by default.<br>True or false, choose one.<br>To use the smile model, set it to true when initializing FacePassHandler and pass the model. Alternatively, pass only the model during initialization, then enable it through the setConfig interface later. |

<table>
<tr><td></td><td></td><td>rcAttributeAndOcclusionMode</td><td>int</td><td>Default mode 0 for occlusion and properties.<br><br>You can choose from 5 modes: {0,1,2,3,4}. The default is mode 0.<br><br>The entry mode has only two options: 0 (entry allowed when blocked) and 2 (entry not allowed when blocked).<br><br><table><tr><th>pattern</th><th>Dependent model</th><th>Usage scenario</th></tr><tr><td>0</td><td>Not<br>have</td><td>Allow recognition or database entry of obscured faces</td></tr><tr><td>1</td><td>Attribute model</td><td>Return face attribute information</td></tr><tr><td>2</td><td>Occlusion model</td><td>Returns the occlusion information of the face that is not occluded when it is guaranteed to be recognized or stored in the database.</td></tr><tr><td>3</td><td>Occlusion model and attribute model</td><td>Allow faces with obscured recognition<br><br>Return face occlusion and attribute information per frame<br><br>Return the occlusion and attribute information of the face returned for recognition</td></tr><tr><td>4</td><td>Occlusion model and attribute model</td><td>Ensure the face is not obscured during recognition<br><br>Return face occlusion and attribute information per frame</td></tr></table><br>pattern 0<br><br>This mode, which does not rely on any model, sets FacePassHandler to 0 during initialization. It is the fastest mode and does not return valid occlusion or attribute information, making it suitable for high-throughput scenarios.<br><br>Select this mode for warehousing. Obstruction is allowed!<br><br>pattern 1<br><br>To use the attribute model, set it to 1 during FacePassHandler initialization and pass the model. Alternatively, pass only the model during initialization, then enable its functionality through the setConfig interface later.<br>When this feature is enabled, the feedFrame/feedFrameRGBIR interface will return facial attribute data.<br>Select this mode during warehousing, which is equivalent to mode 0. Obstruction is allowed during warehousing!<br><br>pattern 2：<br><br>You can either set the value to 2 and pass the model when initializing FacePassHandler, or pass only the model and enable its features later through the setConfig interface.<br>The occlusion model consumes computational resources. When it is not necessary to ensure that the face being recognized is a full face, the model can be omitted to enhance the overall performance of the algorithm system.<br>Select this mode for warehousing. Blocking is not allowed!<br><br>Mode 3 and Mode 4<br><br>You can use the occlusion model and attribute model to set the value to 3/4 and pass both models when initializing FacePassHandler. Alternatively, you can pass only the model during initialization and enable its features later through the setConfig interface.<br><br>When this feature is enabled, the feedFrame/feedFrameRGBIR interface will return facial occlusion and attribute data to determine mask usage compliance. Other features are available for users to explore.<br><br>Select mode 3/4 when entering inventory, equivalent to mode 0. Obstruction is allowed!<br><br>The key difference between Mode 3 and Mode 4 is that Mode 3 allows partial facial recognition instead of full-face recognition.</td></tr>
<tr><td></td><td></td><td>dynamicLocalGroupEnabled</td><td>bool</td><td>The dynamic base database feature is disabled by default.<br><br>True or false, choose one.<br><br>After enabling this feature, refer to the relevant documentation for usage.</td></tr>
<tr><td></td><td></td><td>eyeocEnabled</td><td>bool</td><td>Enabling open eye model<br><br>True or false, choose one.</td></tr>
<tr><td></td><td></td><td>smallSearchEnabled</td><td>bool</td><td>Enable recognition of small models<br><br>True or false, choose one.<br><br>The small-hour model will only be activated when both re-search and live processes are active simultaneously.<br><br>Recognizing small models consumes computational resources. In scenarios where small models are unnecessary, the liveness switch can be disabled to enhance overall algorithm performance.</td></tr>
</table>

| | | | | |
|---|---|---|---|---|
| Quality judgment | Quality assessment comprises a set of metrics, including minimum face size, angle, blur, and lighting.<br>The function of quality assessment is to evaluate the quality score of captured facial images, which determines whether to proceed with facial recognition.<br>Poor-quality images negatively impact both recognition and liveness detection.<br>The more rigorous the quality judgment, the more beneficial it is for identification. However, the stricter the judgment, the longer the time spent on detection (as substandard frames are discarded), which also represents a trade-off.<br>We provide a default set of parameters, but also allow custom modifications for each parameter value in the config. | faceMinThreshold | int | Minimum face size.<br>Indicates the minimum face size allowed for recognition.<br>The Int value ranges from 0 to 512. While theoretically both extremes are acceptable, such extreme values are strongly discouraged in practical applications. FaceMin performs size comparison after detecting faces, applying to both length and width dimensions. This means both must meet the following criteria simultaneously:<br>• width >= faceMin<br>• height >= faceMin<br>If not satisfied, this frame will be discarded and the next frame will be waited for.<br>FaceMin directly affects the recognition distance: a smaller faceMin allows faces to be farther from the camera, but reduces the accuracy of the algorithm and the detection of live faces; a larger faceMin improves the algorithm's performance, but requires the user to move closer to the camera.<br>Since this algorithm's performance is hard to quantify, we've adopted a comprehensive strategy:<br>We recommend clients to maintain a faceMin value higher than Megvii's suggested threshold to ensure basic recognition performance. If reducing faceMin is necessary to enhance distance detection, they should compensate by optimizing the overall recognition experience, such as decreasing the database size or improving database quality.<br>Currently, Megvii recommends a value of 150, which generally ensures reliable recognition performance.<br>The relationship between faceMin and recognition distance (based on standard calibration of mainstream 720p cameras):<br>**faceMin** / **Recognition distance**: 100 / 1m; 125 / 0.8m; 150 / 0.5m<br>*Note: Actual performance depends on factors such as camera lens parameters, focal length, and resolution. Please refer to the actual results. |
| | | poseThreshold | FacePassPose | Face angle threshold.<br>The angle is defined as the deviation between a face and a 'fully frontal' position across three axes, with a larger deviation indicating a greater angular difference.<br>The breakdown is divided into three dimensions:<br>**Broad heading** / **Subclass** / **Recommended threshold** / **Remarks**: pose / Roll / 30° / Angle of rotation; Pitch / 30° / Vertical angle; Yaw / 30° / Horizontal angulation<br>Angle threshold: Use angle values for all three dimensions (range: [0,90]) without decimal restrictions. Avoid extreme values at both ends.<br>The recommended angle range is between 20° and 30°. A wider angle means a more 'lenient' restriction, making it easier to pass quality checks. This results in faster successful facial capture and transmission to the terminal for recognition. However, the image quality may also decline, which could negatively impact recognition accuracy, potentially causing missed or misidentified results.<br>The recommended threshold is 30°. Values below 35° are considered acceptable, while higher values may affect recognition accuracy. Start with 30° as the initial value and adjust it downward if you find the precision insufficient. |
| | | lmkoccThreshold | FacePassLmkOcc | Face unobstructed threshold<br>Values above this threshold indicate unobstructed, with detailed breakdowns<br>**Broad heading** / **Subclass** / **Default threshold**: lmkocc / eye / 0.25; nose / 0.30; mouth / 0.25<br>The threshold range is [default threshold, 1.0]. The higher the threshold, the more likely it is to be mistaken as occlusion. When it is necessary to ensure that the face to be recognized is the full face, the target judged as occlusion will not be recognized!**This threshold does not need to be configured by the user**.<br>For the scene of entering the database,when rcAttributeAndOcclusionMode=2, objects larger than this threshold are considered occluded and are not allowed to enter the database! |
| | | edgefacecompThreshold | float | Face edge integrity threshold<br>When the face is at the image edge, the area of the face within the image is a percentage of the total face area, ranging from 0 to 1.<br>**Classify** / **Recommended threshold**: edgefacecomp / 0.75<br>The recommended threshold is 0.75. Scores above 0.75 are considered passed, while scores below 0.75 indicate insufficient facial integrity.<br>A higher threshold improves the completeness of faces for recognition and database entry, enhancing feature extraction. Therefore, it is not recommended to set the threshold too low. Users are advised to use the recommended threshold. |

<table>
<tr><td>blurThreshold</td><td>float</td><td>Face blur threshold.<br><br>Each face has a blur value, and the higher the blur value, the greater the blur.<br><br>There are many reasons for blurring, such as motion or out-of-focus. Blurred images can affect recognition and liveness, so they need to be limited.<br><br>A Float threshold with a range of [0,1], allowing unlimited decimal places and not recommended for extreme values.<br><br><table><tr><th>Classify</th><th>Recommended threshold</th></tr><tr><td>blur</td><td>0.8</td></tr></table><br>The current recommended value is 0.8. Values below 0.8 are considered acceptable, while those above 0.8 are deemed too vague and should not be used for recognition.<br><br>The smaller the Blur threshold, the stricter the system is in determining face blurring, which is most beneficial for recognition. However, due to the strict detection, the pass rate decreases, potentially increasing the number of detections and extending the processing time. In practice, if you encounter blurring issues, you can adjust the settings as needed.</td></tr>
<tr><td>landmarkThreshold</td><td>float</td><td>Face threshold<br><br>The face threshold is used to determine whether the target is a face. The threshold is a Float type with a range of [0,1]. The recommended threshold is 0.7.This threshold does not need to be configured by the user by default.<br><br><table><tr><th>Classify</th><th>Recommended threshold</th></tr><tr><td>landmark</td><td>0.7</td></tr></table><br>When setting higher facial recognition thresholds during database entry, stricter criteria ensure more authentic faces are approved. However, this heightened scrutiny reduces acceptance rates for photos with beauty filters or makeup. Conversely, lower thresholds increase the risk of low-authenticity entries, making it harder to match them with real faces.</td></tr>
<tr><td>facerectThreshold</td><td>float</td><td>Face detection box threshold<br><br>The face detection box threshold is used for pre-identification. If the value is greater than this threshold, a face is detected in the box; if less, no box is output. The threshold range is [0,1], and 0.3 is recommended.<br><br><table><tr><th>Classify</th><th>Recommended threshold</th></tr><tr><td>facerect</td><td>0.3</td></tr></table><br>If the threshold is set too high, it may fail to detect faces. If set too low, it may cause false positives, i.e., non-face objects may be misidentified as faces.</td></tr>
<tr><td>lowBrightnessThreshold</td><td>float</td><td rowspan="2">Average face illumination threshold.<br><br>Average illuminance refers to the brightness of a face after removing hair, accessories, and other impurities. A higher value indicates brighter, but it must not exceed 255.<br><br>The corresponding threshold is Float, ranging from [0,255]. The default recommended range is [70,210]. Values within this range are considered acceptable. Values below 70 are too dark, and values above 220 are too bright, both of which may affect recognition results.<br><br><table><tr><th>Classify</th><th>Recommended threshold</th></tr><tr><td>lowBrightness</td><td>70</td></tr><tr><td>highBrightness</td><td>220</td></tr></table><br>If you adjust the brightness threshold for ambient lighting, make sure to synchronize the recognition results.</td></tr>
<tr><td>highBrightnessThreshold</td><td>float</td></tr>
<tr><td>brightnessSTDThreshold</td><td>float</td><td rowspan="2">Face illumination standard deviation threshold.<br><br>The standard deviation of illuminance is a metric for assessing face illumination contrast, with values ranging from 0 to 255. A higher value indicates stronger contrast.<br><br>The threshold is defined as Float, ranging from 0 to 255. The brightnessSTDThreshold sets the upper limit for the standard deviation threshold. A higher value allows more lenient processing, permitting high-contrast images like yin-yang faces that may cause recognition issues. Conversely, a lower value imposes stricter requirements, but this compromises image quality, thereby reducing detection success rates and efficiency.<br><br>brightnessSTDThresholdLow is the minimum standard deviation of face brightness in the database interface (addFace/extractFeature), which is used to filter out some poor quality faces. It is recommended to take the value [10,80] in the database, and the calculation is passed within this range.<br><br><table><tr><th>Classify</th><th>Recommended threshold for inventory entry</th><th>Non-inventory recommendation threshold</th></tr><tr><td>brightnessSTD</td><td>80</td><td>80</td></tr><tr><td>brightnessSTDLow</td><td>10</td><td>Not<br>have</td></tr></table></td></tr>
<tr><td>brightnessSTDThresholdLow</td><td>float</td></tr>
</table>

| | | | | |
|---|---|---|---|---|
| Business parameters | The SDK also provides parameter configuration for specific business logic, allowing users to customize certain operations. | retryCount | int | Number of retries when recognition fails.<br><br>FacePassFaceint type, <=0, theoretically no ceiling. If it is large enough, it is equivalent to constant retrying, unless the business layer is interrupted or track is not appearing and other logic ends the retrying.<br><br>Retries are always performed for a specific trackId. For the detailed logic, refer to the trackId logic explanation in FacePassHandler.<br><br>The default setting is version 2, meaning the system will stop recognizing the current trackId if two consecutive retries fail. This implies that even if the person associated with the trackId remains in the frame, FacePassDetectionResult will only return the 'display data' from FaceList, without providing the 'recognition data' from message or imageList for identification.<br><br>If users need to customize modifications, it is generally recommended that only two types of modifications are required:<br><br>1. It is believed that the value should be changed to less than 3 to reduce the number of retries and achieve a faster average recognition speed, but the probability of affecting the recognition rate is not significant according to the current experimental results.<br><br>2. If the client prefers to control the retry logic for recognition, they can set a sufficiently large number, such as 99999. This means that as long as a trackId appears on the screen, the system will retry continuously if it fails. However, the client can also implement custom logic (e.g., timeout) to interrupt the retry process. |
| | | fileRootPath | String | Enter a valid file directory path for the root directory of the storage database. |
| | | maxFaceEnable | bool | Maximum face enable switch is off by default.<br><br>True, enables maximum face detection. If multiple face boxes are detected in the same frame, only the largest face is sent for recognition.<br><br>false, disable maximum face detection. If multiple face boxes are detected in the same frame and all pass the quality filter, the system will process all faces simultaneously. |
| | | detectMode | int | SDK detection mode:<br><br>0: Normal detection mode;<br><br>1: Small face detection mode (long-range detection mode);<br><br>2: Small Face Detection Mode (Ultra-Far Detection Mode);<br><br>The default is normal detection mode. The farther the distance, the slower the detection speed. |
| model parameter | The algorithm provides algorithm models for different modules, which are initialized into FacePassModel and then specified as parameters to FacePassConfig. | detectRectModel | FacePassModel | A model for face detection, initialized as an instance through FacePassModel |
| | | poseBlurModel | FacePassModel | A model for evaluating face angle and face blur quality, initialized as an instance through FacePassModel |
| | | searchModel | FacePassModel | A model for face recognition, initialized as an instance through FacePassModel |
| | | livenessModel | FacePassModel | The model for face liveness detection is initialized as an instance with FacePassModel, and livenessEnabled must be set to true. |
| | | rgbIrLivenessModel | FacePassModel | The model for human face infrared liveness detection is initialized as an instance through FacePassModel, requiring rgbIrLivenessEnabled to be set to true simultaneously. |
| | | detectModel | FacePassModel | A model for face detection, initialized as an instance through FacePassModel |
| | | landmarkModel | FacePassModel | A model for obtaining facial keypoints, initialized as an instance through FacePassModel |
| | | ageGenderModel | FacePassModel | Model for age and sex |
| | | smileModel | FacePassModel | The model for the Smile Index requires setting smileEnabled to true simultaneously. |
| | | livenessGPUCache | FacePassModel | This model is for GPU acceleration. When using a CPU live model, pass null. When using a GPU live model, you must pass the acceleration cache model. |
| | | occlusionFilterModel | FacePassModel | The model for face occlusion detection is initialized as an instance through FacePassModel and requires configuration.<br>The rcAttributeAndOcclusionMode value is 2/3/4. For specific definitions, refer to rcAttributeAndOcclusionMode. |
| | | rcAttributeModel | FacePassModel | An attribute model for face detection, initialized as an instance through FacePassModel. Settings are required.<br>If rcAttributeAndOcclusionMode is 1/3/4, it returns face attribute information. For specific definitions, refer to rcAttributeAndOcclusionMode |
| | | eyeocModel | FacePassModel | Model for closed eye |
| | | smallSearchModel | FacePassModel | A small recognition model for enhancing attack prevention |

# member function

## FacePassConfig

The first constructor, Config, initializes the interface without requiring any parameters, and then configures them as needed.

```
public FacePassConfig()
```

### parameter declaration

not have

### returned value

FacePassConfig instance.

### Example

```
/* CPU */
config = new FacePassConfig();
config.poseBlurModel = poseBlurModel;
config.livenessModel = livenessModel;
config.searchModel = searchModel;
config.detectModel = detectModel;
config.detectRectModel = detectRectModel;
config.landmarkModel = landmarkModel;
config.smileModel = smileModel;
config.ageGenderModel = ageGenderModel;
config.occlusionFilterModel = occlusionFilterModel;
config.rcAttributeModel = rcAttributeModel;

config.livenessEnabled = true;
config.rgbIrLivenessEnabled = false;
config.smileEnabled = true;
config.maxFaceEnabled = true;
config.rcAttributeAndOcclusionMode = 0;

config.searchThreshold = searchThreshold;
config.livenessThreshold = livenessThreshold;
config.faceMinThreshold = faceMinThreshold;
config.poseThreshold = new FacePassPose(roll, pitch, yaw);
config.blurThreshold = blurThreshold;
config.lowBrightnessThreshold = lowBrightnessThreshold;
config.highBrightnessThreshold = highBrightnessThreshold;
config.brightnessSTDThreshold = brightnessSTDThreshold;

config.retryCount = retryCount;
config.fileRootPath = fileRootPath;
config.detectMode = detectMode;
```

The second constructor, Config creation, takes the generated config as a parameter to setAddFaceConfig, passing only the threshold for database entry to specify the entry threshold.

```
public FacePassConfig(int faceMinThreshold,
                                        float roll,
                                        float pitch,
                                        float yaw,
                                        float eye,
                                        float nose,
                                        float mouth,
                                        float blurThreshold,
                                        float landmarkThreshold,
                      float edgefacecompThreshold,
                                        float lowBrightnessThreshold,
                                        float highBrightnessThreshold,
                      float brightnessSTDThreshold,
                                        float brightnessSTDThresholdLow)
```

**Parameter Declaration**

| Parameter type | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| faceMinThreshold | int | Required | See variable description |
| roll | float | Required | See variable description |
| pitch | float | Required | See variable description |
| yaw | float | Required | See variable description |
| eye | float | Required | See variable description |
| nose | float | Required | See variable description |
| mouth | float | Required | See variable description |
| blurThreshold | float | Required | See variable description |
| landmarkThreshold | float | Required | See variable description |
| edgefacecompThreshold | float | Required | See variable description |

| | | | |
|---|---|---|---|
| lowBrightnessThreshold | float | Required | See variable description |
| highBrightnessThreshold | float | Required | See variable description |
| brightnessSTDThreshold | float | Required | See variable description |
| brightnessSTDThresholdLow | float | Required | See variable description |

### returned value

FacePassConfig instance.

## FacePassDetectionResult

### Summary

The class of the image detection result.

It is primarily used for the return results of the feedFrame interface in FacePassHandler.

Fundamentally, this data structure represents 'one frame detection corresponding to a face image and its associated information,' which consists of two components:

1. faceList: An array of FacePassFaces (see the corresponding class for definition). This array provides users with facial detection information such as face bounding boxes, serving as the basis for GUI display and other upper-layer operations.

2.message: A message is an encrypted string. In offline mode, it is passed to the recognize interface of FacePassHandler.

3. images: This is an array of FacePassImages, a list of cropped facial images that have passed quality filters. In offline mode, these facial masks are not used for recognition (all recognition uses the original images). The images store the mask information for facial recognition, thus corresponding to the message. If the message is empty, the images will also be empty. If the message is not empty, the images contain the mask information for facial recognition.

The logic related to detect will also be explained in the feedFrame interface.

### Member variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| message | byte[] | The string for facial recognition is encrypted and generated by the algorithm based on the detection results, containing some information fields required for facial recognition. If no facial information is available for recognition, such as the current frame's face being of poor quality, this field length is 0. |
| faceList | FacePassFace[] | The FacePassFace array contains facial detection results for customer use, as specified in the FacePassFace class definition.<br>In simple terms, it contains track_id (an ID identifying the same person) and rect (the face bounding box). The client can use this information for GUI display. If no face is detected, the array is empty. |
| images | FacePassImage[] | The FacePassImage array contains a list of face images that have passed quality filters and been cropped. For details, see the FacePassImage class definition.<br>In simple terms, it refers to the data of cropped images based on individual faces, including original data streams, dimensions (height and width), face bounding boxes, keyframe bounding boxes, rotation control parameters, and other related fields. trackId, etc. Users can remove objects by selecting the frame.<br>If no face is detected, this array is empty. |

### Member Function

not have 。

## FacePassFace

### Summary

Face information.

Provided to customers for real-time display of face detection results, it appears only in the FacePassDetectionResult field of the feedFrame.

### Member Variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| trackId | long | A long ID used as a track identifier to mark a person in continuous motion.<br>To clarify, a person may have multiple consecutive frames, but not all of them are sent to the backend for comparison, as that would consume excessive resources.<br>The algorithm intelligently identifies which faces belong to the same individual and assigns them identical track IDs.<br>While each frame is independent, the algorithm synthesizes multi-frame data through track_id to achieve facial recognition. For example, if a frame under a specific track_id passes recognition, the access control system will grant entry, eliminating the need for subsequent frames under that track_id to be re-recognized. |
| rect | FacePassRect | Face region, see the FacePassRect class definition.<br>The coordinates of the upper left and lower right points are used to depict the position of the face frame, which is generally used in GUI display. |
| pose | FacePassPose | The facial orientation is represented by three angles: roll, pitch, and yaw, with values ranging from [-90°,90°] in float format (see FacePassPose class definition). If the facial orientation deviates excessively, the system prompts the user via GUI to face the camera directly. |
| blur | float | Face blur level (see FacePassConfig definition for details). A float between [0,1] where higher values result in greater blurring. |
| rcAttr | FacePassRCAttribute | The attribute type value, as defined in the FacePassRCAttribute class, is invalid when facePassFeedFrameErrorCode is not 0. |
| lmkoccsta | FacePassLmkOccStatus | The occlusion status value, as defined in the FacePassLmkOccStatus class. When FacePassLmkOccStatus's valid is 0, this property is invalid. |
| edgefacecomp | float | Face completeness (0,1]:1.0 if the face is inside the image, and the ratio of the face area inside the image to the total face area if the face is on the edge. |
| landmarkscore | float | face landmark confidence,landmarkThreshold,between |
| rectscore | float | The confidence level of the face detection box, [facerectThreshold, 1] |
| smile | float | The Smile Index, a value between 0 and 1, indicates the higher the probability of a smile emoji. The smile detection feature is activated only when the smileModel is passed; otherwise, it remains disabled. |
| facePassFeedFrameErrorCode | int | feedFrame error code. When multiple thresholds are exceeded, only one error code is returned. The specific correspondence is as follows:<br>0: "FEEDFRAME_OK_CODE", no errors occurred during the feedFrame process;<br>1: "FEEDFRAME_ROLL_ERROR_CODE", the roll value of face quality detection during feedFrame exceeds the threshold;<br>2: "FEEDFRAME_PITCH_ERROR_CODE", the pitch value detected during face quality assessment in the feed frame process exceeds the threshold;<br>3: "FEEDFRAME_YAW_ERROR_CODE", the yaw value of face quality detection during feedFrame exceeds the threshold;<br>4: "FEEDFRAME_BLUR_ERROR_CODE", the blur value of face quality detection during the feedFrame process exceeds the threshold;<br>5: "FEEDFRAME_MINFACE_ERROR_CODE", the minface value for face quality detection during the feedFrame process falls below the threshold;<br>6: "FEEDFRAME_HALFFACE_ERROR_CODE" indicates that the halfface value for facial quality detection during the feedFrame process falls below the threshold.<br>7: "FEEDFRAME_OCCLUSION_ERROR_CODE", the face quality detection during feedFrame exceeds the threshold due to occlusion;<br>8: "FEEDFRAME_BRIGHTNESS_ERROR_CODE" -The brightness value detected during face quality assessment in the feed frame process falls<br>outside the threshold range. 9: "FEEDFRAME_UNKNOWN_ERROR_CODE" -An unknown error occurred during the feed frame process. |
| eyeclosed | FacePassEyeClosed | The closed-eye state returned by the closed-eye model, as defined in the FacePassEyeClosed class. This property is invalid when FacePassEyeClosed's valid is false. |

## Member Function

not have 。

# FacePassDetectFace

## Summary

The facial information is exclusively contained in the FacePassDetectFacesResult output from the detectFaces function.

## Member Variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| rect | FacePassRect | Face region, see the FacePassRect class definition.<br>The coordinates of the upper left and lower right points are used to depict the position of the face frame, which is generally used in GUI display. |
| pose | FacePassPose | The facial orientation is represented by three angles: roll, pitch, and yaw, with values ranging from [-90°,90°] in float format (see FacePassPose class definition). If the facial orientation deviates excessively, the system prompts the user via GUI to face the camera directly. |
| blur | float | Face blur level (see FacePassConfig definition for details). A float between [0,1] where higher values result in greater blurring. |
| edgefacecomp | float | Face completeness (0,1]:1.0 if the face is inside the image, and the ratio of the face area inside the image to the total face area if the face is on the edge. |
| brightness | float | The brightness value of the face ranges from 0 to 255, with higher values indicating brighter faces. |
| deviation | float | The standard deviation of face brightness, ranging from 0 to 255. A higher value indicates greater face contrast. |
| lData | byte[] | The lData facial data can be used to extract feature values of the corresponding face in the current image through the extractFeatureWithLData interface. |

## Member Function

not have 。

# FacePassImage

## Summary

Face image category.

This SDK class handles image raw data, primarily containing unprocessed data, while also providing control parameters for dimensions and rotation.

The client initializes the system by returning data via the camera, then generates a FacePassImage object as a parameter to the handler's feedFrame function to avoid cumbersome parameter passing.

## Member Variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| image | byte[] | The raw image data depends on the actual size of the image. |
| width | int | Image width in pixels. In the "Camera Frame Processing" scenario, the client must ensure the value is correct to avoid algorithm processing issues. |
| height | int | Image height in pixels. The client must ensure the value is correct in the Camera Frame Processing scenario, otherwise it may cause algorithm processing issues. |
| facePassImag eRo-tation | int | The image rotation angle values differ from the forward orientation (face-up direction). Only four valid DEG values (DEG0, DEG 90, DEG180, DEG270) defined in FacePassImageRotation represent the rotation angles of 0°,90°,180°, and 270°.<br>This parameter requires user control. If the image is rotated at an incorrect angle, the system may fail to detect the face. |
| facePassImag eType | int | Supported image format types: 0: "NV21", 1: "GRAY", 2: "RGB".<br>Ensure the parameters are correct to avoid algorithm processing issues. |
| trackId | long | A long ID used as a track identifier to mark a person in continuous motion.<br>Enter the "frame detection return" scenario, where the algorithm assigns values to each object based on the detection results. |
| rect | FacePassRect | Face region, see the FacePassRect class definition.<br>The coordinates of the upper-left and lower-right points define the facial bounding box on the image. |
| rect_ex | FacePassRect | Face mask frame. See the FacePassRect class definition.<br>The coordinates of the upper-left and lower-right points define the facial bounding box on the image. |
| pose | FacePassPose | The facial orientation is represented by three angles: roll, pitch, and yaw, with values ranging from [-90°,90°] in float format (see FacePassPose class definition). If the facial orientation deviates excessively, the system prompts the user via GUI to face the camera directly. |
| blur | float | Face blur level (see FacePassConfig definition for details). A float between [0,1] where higher values result in greater blurring. |
| edgefacecomp | float | Face completeness (0,1]:1.0 if the face is inside the image, and the ratio of the face area inside the image to the total face area if the face is on the edge. |

| | | |
|---|---|---|
| brightness | float | The brightness value of the face ranges from 0 to 255, with higher values indicating brighter faces. |
| brightness_std | float | The standard deviation of face brightness, ranging from 0 to 255. A higher value indicates greater face contrast. |
| rcAttr | FacePassRCAttribute | Send the attribute information of the recognized face, as defined in FacePassRCAttribute. |
| lmkoccsta | FacePassLmkOccStatus | The occlusion state value, as defined in the FacePassLmkOccStatus class. |
| eye_closed | FacePassEyeClosed | State value for closed eyes. See the FacePassEyeClosed class definition for details. |

### Member Function

### FacePassImage

constructed function 。

```
public FacePassImage(byte[] image, int width, int height, int facePassImageRotation, int facePassImageType)
```

#### parameter declaration

| Parameter type | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| image | byte[] | Required | See variable description |
| width | int | Required | See variable description |
| height | int | Required | See variable description |
| facePassImageRotation | int | Required | See variable description |
| facePassImageType | int | Required | See variable description |

### returned value

FacePassImage instance.

## FacePassPose

## Summary

Face angle category.

Corresponding to the three axes of the 3D world, divided into three parameters, as shown in the table below.

## Member variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| roll | float | The rotation angle around the nose ranges from-90 to +90, where 0 indicates a frontal face, clockwise rotation is positive, and counterclockwise rotation is negative. If the angle exceeds 45°, the algorithm may fail to detect the face. |
| pitch | float | The pitch angle ranges from-90 to +90, where 0 indicates a frontal face, with a positive value for looking up and a negative value for looking down. If the angle exceeds 45°, the algorithm may fail to detect the face. |
| yaw | float | The left-right tilt angle ranges from-90 to +90, where 0 indicates a frontal face, left rotation is positive, and right rotation is negative. If the angle exceeds 45°, the algorithm may fail to detect the face. |

### Member Function

### FacePassPose

constructed function 。

```
FacePassPose FacePassPose(float roll, float pitch, float yaw);
```

parameter declaration

| Parameter type | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| roll | float | Required | See variable description |
| pitch | float | Required | See variable description |
| yaw | float | Required | See variable description |

### returned value

FacePassPose instance.

## FacePassLmkOcc

### Summary

Unobstructed face.

The three parameters are eye, nose, and mouth, as detailed in the table below.

### Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| eye | float | The unmasked score ranges from 0 to 1.0 |
| nose | float | The score for an uncovered nose, ranging from 0 to 1.0 |
| mouth | float | The score for an uncovered mouth, ranging from 0 to 1.0 |

### member function

### FacePassLmkOcc

constructed function 。

```
FacePassLmkOcc FacePassLmkOcc(float eye, float nose, float mouth);
```

parameter declaration

| Parameter type | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| eye | float | Required | See variable description |
| nose | float | Required | See variable description |
| mouth | float | Required | See variable description |

### returned value

FacePassLmkOcc instance.

## FacePassLmkOccStatus

### Summary

Face occlusion state class.

Valid for four occlusion parameters: valid, eyeocc, noseocc, and mouthocc. See the table below for details.

## Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| valid | boolean | Is the occlusion property value valid? True means valid. |
| eyeocc | boolean | Is one eye covered? A covered eye means it is blocked. true indicates a blockage. |
| noseocc | boolean | Whether the nose is blocked. true means it is blocked. |
| mouthocc | boolean | Whether the mouth is covered. true means it is covered. |

## Member Function

No. The constructor is used only within the SDK.

# FacePassEyeClosed

## Summary

Closed-eye state class.

valid, left_eye, right_eye: three parameters for closing the eyes. See the table below for details.

## Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| valid | boolean | Is the occlusion property value valid? True means valid. |
| left_eye | boolean | Left eye closed. True indicates the left eye is closed. |
| right_eye | boolean | Right eye closed. True indicates the right eye is closed. |

## Member Function

No. The constructor is used only within the SDK.

# FacePassRect

## Summary

The rectangular region class corresponding to the face location.

## Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| left | Int | Left face boundary. |
| top | Int | The boundary of the human face. |
| right | Int | Right edge of the face. |
| bottom | Int | The bottom edge of the face. |

### Member Function

constructed function：

```
FacePassRect FacePassRect(int left, int top, int right, int bottom);
```

**parameter declaration**

| Parameter type | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| left | int | Required | See variable description |
| top | int | Required | See variable description |
| right | int | Required | See variable description |
| bottom | int | Required | See variable description |

### returned value

### FacePassRect

FacePassRect instance.

## FacePassRecognitionResult

### Summary

The data class of face recognition results.

The client invokes the member function recognize of FacePassHandler (commonly known as the facial recognition interface) and returns the result from this class.

### Member Variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| trackId | long | A long ID used as a track identifier to mark a person in continuous motion. |
| faceToken | byte[] | A visible string of 24 to 100 characters, serving as a unique identifier generated after adding a face. It is used for core operations like face addition, deletion, and database binding. Generally, faceToken represents a face in the database.<br>The facial recognition process consists of two components: facial recognition (Recognition) and liveness detection (Liveness).<br>1. Face recognition: It identifies the most similar face to a target within a group, represented by a faceToken.<br>2. Livescreen detection: Independent of Group, verifies whether the target is a real person rather than a paper or electronic screen-based attack.<br>The purpose of returning faceToken is to enable the client to generate a complete facial recognition record, which is then fed back to the customer and logged.<br>Returns an empty string if no face is detected. |
| rect | FPRect | Face detection box |
| image | FacePassImage | Save the information for the recognition image, such as width, height, orientation, and type. Other information is invalid. |
| detail | FacePassRecognitionDetail | See the class definition of FacePassRecognitionDetail for details.<br>This represents the actual details of a face recognition process, including the scores and thresholds for search, liveness, and small recognition. |
| featureData | byte[] | Feature data typically measures 256,512,1024, or 2048 bytes (varies by feature model, as per actual specifications). |

| | | |
|---|---|---|
| facePassRecognitionState | int | The result state of face recognition corresponds to the following specific relationship:<br>0: "RECOGNITION PASS", indicating successful facial recognition, where both search and liveness thresholds were met.<br>1: "RECOGNITION_RETRY" indicates a failed recognition attempt, which may be due to either search error (searchErrorCode) or liveness error (livenessErrorCode) not meeting thresholds. The system will retry the recognition before the user-defined retryCount is exhausted.<br>2: "RECOGNITION_RETRY_EXPIRED" indicates failed recognition when the system fails to identify the same track_id multiple times (retryCount). The recognition process is then terminated for that track_id. Notably, exceptions thrown by recognize due to unmet thresholds (e.g., failed liveness check or insufficient recognition score) are excluded from this count.<br>3: "RECOGNITION TRACK MISSING" – The system is in recognition mode. If the face is partially obscured or the user triggers the reset function, it may display a track ID loss alert. However, this condition rarely occurs as the current recognition model operates at high speed.<br>Users can perform subsequent logical operations based on the returned value or display it via GUI. |
| searchErrorCode | int | During the recognize process, only the first failed search error code is returned, with the following<br>mapping: 0: "SEARCH_OK_CODE", indicating the search score passed the threshold.<br>1: "SEARCH_UNPASS_ERROR_CODE", when the search score during recognition falls below the threshold;<br>2: "SEARCH_UNKNOWN_ERROR_CODE" indicates an unknown error occurred during the recognition process. |
| livenessErrorCode | int | During the recognition process, only the first failure error code of liveness is returned, with the specific correspondence as follows:<br>0: "LIVENESS_OK_CODE", the liveness score during recognition process exceeds the threshold;<br>1: "LIVENESS_IRDETECT_ERROR_CODE" -The liveness detection (irDetect) failed during recognition, likely due to: 1) failed face detection, 2) overlap score below threshold, or 3) pixel frequency (pf) below threshold.<br>2: "LIVENESS_UNPASS_ERROR_CODE", the liveness score is below the threshold during recognition;<br>3: "LIVENESS_UNKNOWN_ERROR_CODE" -An unknown error occurred during liveness recognition. |
| smallSearchErrorCode | int | The error codes of smallsearch during recognition, with their specific mappings as follows:<br>0: "SMALL_SEARCH_OK_CODE", during recognition, the smallsearch score exceeds the threshold;<br>1: "SMALL_SEARCH_UNPASS_ERROR_CODE", when the smallsearch score during recognition falls below the threshold;<br>2: "SMALL_SEARCH_UNKNOWN_ERROR_CODE" -An unknown error occurred during the smallsearch recognition process.<br>Note: If smallSearchEnabled is false or search/active is not expired, the return value is SMALL_SEARCH_OK_CODE with a score of 100.0 f. |

## Member Function

not have 。

# FacePassRecognitionDetail

## Summary

The class for face comparison details.

A representative set of facial recognition parameters, including scores and thresholds for both search and liveness metrics.

## Member Variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| searchScore | float | Strictly speaking, the facial recognition score is the comparison score of the most similar individual in the group.<br>A float value between 0 and 100, where higher values indicate greater similarity. |
| searchThreshold | float | The facial recognition system uses a threshold: if the searchScore exceeds this threshold, the request is approved; otherwise, it is rejected.<br>A float value between 0 and 100. |
| rcAttr | FacePassRCAttribute | Attribute information, see the FacePassRCAttribute class definition for details. |

| | | |
|---|---|---|
| livenessScore | float | The score of the live detection.<br>A float value between 0.00 and 100.00, where higher values indicate more realistic human behavior, and lower values suggest more inauthentic attacks.<br>100.00: This indicates the live switch is not enabled, and the SDK assigns a score of 100.00 to the live result to avoid being affected by the live threshold.<br>-1.00: Indicates an anomaly in the live detection. For binocular live detection, this may occur when the IR image fails to detect the face or the binocular overlap does not meet the threshold. |
| livenessThreshold | float | The liveness score must exceed the threshold to pass; otherwise, the request is rejected.<br>A float value between 0 and 100. |
| dyInfo | FacePassDynamicLocalGroupInfo | After enabling the dynamic base database, the relevant information returned is specified in the FacePassDynamicLocalGroupInfo class definition. |
| smallSearchScore | float | The score of the small recognition.<br>A float value between 0.00 and 100.00, where higher values indicate greater similarity between IR and RGB.<br>100.00: This indicates that the minor recognition switch is not activated, or the search and live detection criteria are not met. The SDK will assign a live detection score of 100.00 to avoid minor recognition interference.<br>-1.00: indicates an anomaly in the small recognition test. |
| smallSearchThreshold | float | The threshold for the hourly model, where smallSearchScore is a float value between 0 and 100. If it exceeds this threshold, the small recognition is approved; otherwise, it is rejected. |

## Member Function

not have 。

# FacePassAddFaceDetectionResult

## Summary

Indicates the results of face detection for face registration.

This class is exclusively for the return value of FacePassHandler's addFaceDetect method and is intended for reference.

## Member variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| image | FacePassImage | Face images for storage.<br>Returns NULL if no face is detected.<br>If a face is detected but fails the quality check, return NULL.<br>When multiple faces are present, only the largest face's quality will be evaluated. |
| faceList | FacePassFace[] | The FacePassFace array contains facial detection results for customer use, as specified in the FacePassFace class definition.<br>This array contains track_id (an ID identifying the same person) and rect (the face bounding box). The client<br>can use this information for GUI display. If no face is detected, the array is empty, not default, and not NULL. |
| lData | byte[] | The lData corresponding to the face detection result can be used to extract feature values of the corresponding face<br>in the current image through the extractFeatureWithLData interface. If no face is detected, this field is empty. |

## Member Function

not have 。

# FacePassDetectFacesResult

## Summary

The human face detection interface detectFaces returns the detected face results.

## Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| faceList | FacePassDetectFace[] | The FacePassDetectFace array contains face detection results, as detailed in the class definition of FacePassDetectFace. |

## Member Function

not have 。

# FacePassAddFaceResult

## Summary

Register a face and return the result in offline mode.

Refer to the return value description of the addFace function in FacePassHandler.

### Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| faceToken | byte[] | A visible string of 24 to 100 characters, generated by the system after successful database entry, serves as a unique facial identifier (faceID). Users can bind or unbind the database using faceToken, and it also supports face deletion and query operations. |
| facePassRect | FacePassRect | Add the face location after successful face recognition |
| result | int | 0: Success<br>1: No face detected<br>2: Face detected but not approved for quality<br>Other values: Unknown error |
| pose | FacePassPose | The pose information of the face image, as defined by FacePassPose |
| blur | float | The blur value of the face image ranges from 0 to 1.0. A smaller value means the image is clearer and the face quality is higher. |
| brightness | float | The brightness value of the face ranges from 0 to 255. A higher value means the face is brighter. |
| deviation | float | The standard deviation of face brightness, ranging from 0 to 255. A higher value indicates greater face contrast. |
| edgefacecomp | float | Face completeness (0-1): A higher value indicates a more complete face in the image. |
| landmarkscore | float | Face confidence, the higher the value, the more it is considered as a face. If the value is lower than landmarkThreshold, no face is detected! |
| lmkoccsta | FacePassLmkOccStatus | Face occlusion status, reference FacePassLmkOccStatus |

## Member Function

No. The constructor is used only within the SDK.

# FacePassAgeGenderResult

**summary**

Age and sex results.

### Member variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| trackId | long | track ID。 |
| age | float | Age |
| gender | int | 0: Male<br>1: Female |

## Member Function

not have 。

# FacePassCompareResult

## Summary

The return value of the compare interface in FacePassHandler.

Compare two images passed through the interface for a 1:1 match.

The two provided images will undergo face detection. If the liveness detection parameter is enabled, liveness detection will also be performed.

If face detection fails, an error will be reported (comparing non-compliant photos is meaningless, and we cannot guarantee the results' practical value).

If all the above steps are correct, a compre score will be returned to determine whether the test passed.

The score ranges from 0 to 100, where higher values indicate greater similarity (the same person) and lower values indicate less similarity (two different people).

The standard practice involves comparing the score against a threshold, with a commonly recommended threshold of 70.

Note that unlike standard image usage rules, the compare API currently lacks strict quality checks. This is because it serves as a low-level API, and clients often deploy it in complex scenarios where establishing a robust quality assurance system is challenging.

Therefore, our strategy is to provide detailed parameters in the API return values. Users can then customize their quality system requirements (while disregarding our comparison scores).

## Member Variables

| Variable name | Type of variable | Variable declaration |
| --- | --- | --- |
| result | Int | The face comparison result is an Int enumeration:<br>0: Successful comparison, indicating both images have passed facial detection. If the live detection is enabled, both also pass the live verification, resulting in the final score.<br>1: Face detection failed. If either image fails detection, the result is considered unsuccessful. For details, refer to the rect parameter in the FacePassFace data structure (detectionResult1 and detectionResult2). A NULL value indicates failed face detection.<br>2: Liveness detection fails. This occurs only when the parameter livenessEnabled in FacePassHandler's compare interface is set to true. If either image fails facial recognition, the system returns result=2 and stops quality score comparison. |
| score | Float | 1:1 comparison score (0,100].<br>This value is assigned a positive number only when result = 0; otherwise, it defaults to 0.<br>The higher the score, the more similar (the same person), and the lower the score, the less similar (two different people).<br>The standard practice involves comparing the score against a threshold, with a commonly recommended threshold of 70. |

| | | |
|---|---|---|
| compareThreshold | Float | The recommended threshold for 1:1 comparison, a Float value in the range [0,100].<br>If the client does not wish to adopt the SDK's recommended threshold, they may set their own. The general principle is:<br>The pass rate = the number of correct answers / the total number of attempts.<br>The error rate was calculated as the number of times I was correct compared to others divided by the total number of times I was correct compared to others.<br>As the threshold increases, the pass rate and false recognition rate decrease. Intuitively, this means you are less likely to fail against yourself but more likely to make fewer mistakes compared to others. Customers can theoretically adjust their usage threshold based on actual needs. If they encounter difficulties, they can consult support staff. |
| detectionResult1 | FacePassFace | The first two parameters of FacePassHandler's compare function, image1 and image2, are the results of face detection. This is a FacePassFace data structure, as detailed in its corresponding definition.<br>For compare, the meaningful parameters are rect, pose, and blur.<br>"rect": Defines the face bounding box position in image1/image2. If detection fails (result = 1), it returns NULL; otherwise, it returns the FacePassRect data structure.<br>The parameters "pose" and "blur" denote angle and blurriness respectively. It should be noted that these quality metrics do not affect the API's output validation. As previously explained, the API's application scenarios are too broad, and many situations involve human oversight with time-based verification, hence the general expectation for the API to adopt more lenient requirements. A typical example is the output image from an ID card reader, which may have subpar quality (with potentially unsatisfactory blur), yet we still aim to pass the final quality check. Therefore, we default to not enforcing strict constraints on angle and blur. For users requiring stricter controls, secondary development can implement higher-level business logic to evaluate these parameters in the output. |
| detectionResult2 | FacePassFace | |
| livenessThreshold | Float | Livescope threshold (Float type), initialized as specified in FacePassConfig configuration. |
| livenessScore1 | Float | The compare function in FacePassHandler processes the liveness detection results of the first two parameters, image1 and image2, which are Float values. A higher value indicates a more authentic human, while a lower value suggests a liveness attack. If either livenessScore1 or livenessScore2 falls below the livenessThreshold, the liveness detection fails, and the result is set to 2. |
| livenessScore2 | Float | |

## Member Function

not have 。

# FacePassSearchResult

## Summary

The data class for image comparison results.

## Member variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| searchScore | float | Compare scores, range (0,100].<br>A higher score indicates greater similarity (the same person), while a lower score indicates less similarity (two different people).<br>When the user's compare1xN interface topK exceeds the minimum database size, the comparison score for the excess portion is-100.0 points. |
| searchThreshold | float | The face comparison threshold ranges from 0.0 to 100.0. |
| faceToken | byte[] | The unique identifier for the face, faceToken.<br>When searchScore is-100.0, faceToken is empty and this value is unavailable. |

### Member Function

not have 。

# FacePassLivenessResult

## Summary

The data class of the recognition results of living organisms.

The client invokes the member function livenessClassify of FacePassHandler and returns the result of this class.

## Member variables

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| trackId | long | A long ID used as a track identifier to mark a person in continuous motion. |
| livenessScore | float | Liveness score, normal range (0.00 to 100.00]. A value of-1.00 indicates that the livenessErrorCode is LIVENESS_IRDETECT_ERROR_CODE. |
| livenessThreshold | float | Living threshold. |
| FacePassLivenessState | int | The result states of live recognition correspond to the following specific relationships:<br>0: "LIVENESSPASS", Liveness recognition passed, meaning the score exceeds the threshold;<br>1: "LIVENESS_RETRY" indicates a failed liveness verification attempt. The system will retry the process before the user's retryCount limit is reached.<br>2: "LIVENESS_RETRY_EXPIRED" -The liveness check failed because the system failed to identify the same track_id multiple times (retryCount). The system will not attempt further identification of this track_id. Exceptions thrown by livenessClassify are excluded from this count.<br>3: "LIVENESS TRACK MISSING" indicates ongoing liveness detection. If the face is partially obscured or the user triggers the reset function, the system may display a track ID loss alert. Given the current high-speed liveness detection model, this status is rarely encountered. Users can either execute subsequent logic operations based on the return value or display the status via the GUI. |
| livenessErrorCode | int | The error code during the liveness process, returning only the first failure error code encountered. The specific mapping is as follows:<br>0: "LIVENESS_OK_CODE", the liveness score passes the threshold;<br>1: "LIVENESS_IRDETECT_ERROR_CODE" — An error occurred during liveness detection (LIVENESS). Possible causes include: failure to detect faces, overlap score below threshold, or pixel frequency (pf) below threshold.<br>2: "LIVENESS_UNPASS_ERROR_CODE", the liveness score falls below the threshold;<br>3: "LIVENESS_UNKNOWN_ERROR_CODE" -An unknown error occurred during the liveness check. |
| rcAttr | FacePassRCAttribute | Attribute information, see the FacePassRCAttribute class definition for details |

## Member Function

not have 。

# FacePassExtractFeatureResult

## Summary

Feature extraction returns the result.

```
public class FacePassExtractFeatureResult {
    public int result;
    public byte[] featureData;
    public FacePassRect rect;
    public FacePassPose pose;
    public float blur;
    public float brightness;
    public float deviation;
    public float landmarkscore;
    public float edgefacecomp;
    public FacePassLmkOccStatus lmkoccsta;
    ...
    ...
}
```

**Member variables**

| Variable name | Type of variable | Variable declaration |
|---|---|---|
| result | int | 0: Success<br>1: No face detected<br>2: Face detected but not approved for quality<br>Other values: Unknown error |
| featureData | byte[] | The number of eigenvalues is usually 256,512,1024, or 2048 bytes. Different recognition models may vary, so refer to the specific model's output. |
| rect | FacePassRect | Face region, see the FacePassRect class definition.<br>The coordinates of the upper left and lower right points are used to depict the position of the face frame, which is generally used in GUI display.<br>Definition of |
| pose | FacePassPose | The pose information of the face image, as defined by FacePassPose |
| blur | float | The blur value of the face image ranges from 0 to 1.0. A smaller value indicates clearer images and higher face quality. |
| brightness | float | The brightness value of the face ranges from 0 to 255. A higher value means the face is brighter. |
| deviation | float | The standard deviation of face brightness, ranging from 0 to 255. A higher value indicates greater face contrast. |
| edgefacecomp | float | Face completeness (0-1): A higher value indicates a more complete face in the image. |
| landmarkscore | float | Face confidence, the higher the value, the more it is considered as a face. If the value is lower than landmarkThreshold, no face is detected! |
| lmkoccsta | FacePassLmkOccStatus | Face occlusion status, reference FacePassLmkOccStatus |

## Member Function

No, the constructor is used only within the SDK.

# FacePassFeatureAppendInfo

### summary

Optional additional information for the feature import interface, including user-defined faceToken (unique facial ID) and faceImage (facial mask).

```
public class FacePassFeatureAppendInfo {
    public String  faceToken;
    public  Bitmap faceImage;
    ...
}
```

**Member variables**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| faceToken | Selectable | String | A unique ID for each face, stored in the same database, must ensure that each face's faceToken is distinct. The string length is limited to 24 to 100 visible characters. If the user provides an empty faceToken or a string shorter than 24 bytes, the parameter will be invalid.<br><br>Invalid. If this parameter is invalid, the SDK will automatically generate a 24-byte faceToken string. |
| faceImage | Selectable | Bitmap | Face removal. Setting it to null means no face removal is applied. Face removal is mainly used for display after successful recognition. Users can choose whether to use it based on their actual scenario. |

## FacePassFeature

### summary

The face feature value.

```
public class FacePassFeature {
    public byte[] featureData;
    ...
}
```

**Member variables**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| featureData | Required | byte[] | Face feature value, which can be directly obtained from the result returned by the extractFeature interface. For details, refer to the description of the extractFeature return result. |

## FacePassRCAttribute

### summary

The model outputs six types of multi-class attributes:

1. Hair: No hair, minimal hair (including baldness), short hair, long hair, unknown, invalid
2. Beard: No beard or barely visible, beard on the lips, mustache, unknown, invalid
3. Hats: None, safety helmet, other hats, unknown, invalid
4. Mask: Mask covers mouth and nose, mask covers mouth, no mask, unknown, ineffective
5. Glasses: None, Sunglasses, Other glasses, Unknown, Invalid
6. Skin color: yellow, white, black, dark, invalid

Invalid indicates that the attribute value is invalid, possibly due to a disabled attribute model switch or an attribute detection error. Unknown means the attribute model cannot determine which category the attribute belongs to.

```
public class FacePassRCAttribute {
    public enum FacePassHairType {
        BALD, LITTLE_HAIR, SHORT_HAIR, LONG_HAIR, UNKNOWN, INVALID
    }
    public enum FacePassBeardType {
        NO_BEARD, MOUSTACHE, WHISKER, UNKNOWN, INVALID
    }
    public enum FacePassHatType {
        NO_HAT, SAFETY_HELMET, OTHERS, UNKNOWN, INVALID
    }
    public enum FacePassRespiratorType {
        COVER_MOUTH_NOSE, COVER_MOUTH, NO_RESPIRATOR, UNKNOWN, INVALID
    }
    public enum FacePassGlassesType {
        NO_GLASSES, GLASSES_WITH_DARK_FRAME, GLASSES_OTHERS, UNKNOWN, INVALID
    }
    public enum FacePassSkinColorType {
        YELLOW, WHITE, BLACK, BROWN, INVALID
    }
    public FacePassHairType  hairType;
    public FacePassBeardType beardType;
    public FacePassHatType   hatType;
    public FacePassRespiratorType  respiratorType;
    public FacePassGlassesType  glassesType;
    public FacePassSkinColorType  skinColorType;
    ...
}
```

**Member variables**

| Parameter name | Parameter type | Parameter declaration |
| --- | --- | --- |
| hairType | FacePassHairType | Hair: No hair, minimal hair (including baldness), short hair, long hair, unknown, invalid |
| beardType | FacePassBeardType | Beard: No beard or barely visible, beard on the lips, mustache, unknown, invalid |
| hatType | FacePassHatType | Cap： No cap, safety helmet, other cap, unknown, invalid |
| respiratorType | FacePassRespiratorType | Mask: Mask covers nose and mouth, mask covers mouth, no mask,unknown, invalid |
| glassesType | FacePassGlassesType | Glasses: None, dark-rimmed transparent glasses, other types of glasses, unknown, invalid |
| skinColorType | FacePassSkinColorType | Skin color: yellow, white, black, dark, invalid |

## FacePassTrackOptions

### summary

Set the recognition score threshold for the specified trackid, including the live score threshold and the small recognition score threshold. This parameter adjusts the threshold for a specific face to be recognized.

```
public class FacePassTrackOptions {
    public long trackId;
    public float searchThreshold;
    public float livenessThreshold;
    public float smallsearchThreshold;
    ...
}
```

**Member variables**

| Parameter name | Parameter type | Parameter declaration |
| --- | --- | --- |
| trackId | long | The trackId for the pending face recognition can be retrieved from the FacePassImage in the FacePassDetectionResult returned by feedFrame/feedFrameRGBIR. |
| searchThreshold | float | trackId[0.00, 100], FacePassConfig |
| livenessThreshold | float | trackId[0.00, 100], FacePassConfig |
| smallsearchThread | float | This trackId corresponds to the liveness threshold for facial recognition, with values ranging from [0.00,100]. Invalid values will use the small recognition threshold set in FacePassConfig. |

## FacePassTrackIdState

### summary

An input parameter of the setMessage interface, used to modify the current trackid's status.

```
public class FacePassTrackIdState {
    public static final int TRACK_ID_RETRY = 0;
    public static final int TRACK_ID_PASS = 1;
    public static final int TRACK_ID_UNPASS = 2;
}
```

**Member variables**

| Parameter name | Parameter type | Parameter declaration |
| --- | --- | --- |
| TRACK_ID_RETRY | int | The trackid status will be reset, the retry count will revert to the user-defined value, and the trackid will continue generating new messages for identification. |
| TRACK_ID_PASS | int | trackidPASStrackidmessage |
| TRACK_ID_UNPASS | int | trackidUNPASStrackidmessage |

## FacePassDynamicLocalGroupInfo

### summary

Identify the dynamic database information returned by the recognition interface to determine whether the current recognition is associated with the dynamic database.

```
public classs FacePassDynamicLocalGroupInfo {
    public boolean enabled;
    public boolean recall;
    public boolean update;
    public float oriSearchScore;
        public int dyUsageState;
}
```

**Member Variables**

| Parameter name | Parameter type | Parameter declaration |
| --- | --- | --- |

| | | |
|---|---|---|
| enabled | boolean | The dynamic base database is enabled only when this value is true; other member variables are invalid. |
| recall | boolean | recall |
| update | boolean | true |
| oriSearchScore | float | The original database recognition score is valid only when recall==true. |
| dyUsageState | int | 0: No dynamic database is triggered<br>1：update=1<br>2：recall=1;<br><0: Unknown issue caused the dynamic database to fail to take effect |

# FacePassHandler

## Summary

FacePassHandler serves as the core engine of the entire client SDK. While it doesn't provide public member variables, it delivers nearly all essential core functions. The following section will detail each Handler member function and explain the underlying technical logic.

## member function

## FacePassHandler

### summary

The constructor requires FacePassConfig for configuration. See the FacePassConfig parameters for details.

To initialize FacePassHandler, you must first call initSDK and ensure the isAvailable function returns true before proceeding.

```
public FacePassHandler(FacePassConfig config) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| config | FacePassConfig | Required | Configuration information, including model version and threshold information. For details, see the FacePassConfig class definition. |

### returned value

FacePassHandler instance object.

### initSDK

### summary

Static function.

The global initialization function of FacePassHandler must be called first. An instance can only be created after initialization is complete.
Note: This function creates an asynchronous thread in the background to perform initialization. The caller must check the success of initialization using the isAvailable function and wait accordingly. Refer to the demo for details.

```
public static void initSDK(final Context context);
public static void initSDK(Context context, final String path);
```

**Parameter Declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|

| | | | |
|---|---|---|---|
| context | Context | Required | Android program context. |
| path | String | Selectable | The path for storing authentication files. The application must have read and write permissions on this path. If no path is specified, the default path is selected. |

### returned value

not have

## isAvailable

### summary

Static function.

This function checks if the SDK has completed initialization. After calling initSDK, you can use this function to monitor progress. Do not perform other operations until you receive a true return value.

```
public static boolean isAvailable();
```

parameter declaration

not have

### returned value

Boolean type, where true indicates initialization is complete

## authPrepare

### summary

Static function.

Authorization preparation. This API must be called after initSDK and before getAuth. After successful authorization, it must still be called.

Note: When using MegSafe soft or hard authorization solutions, you do not need to pay attention to this interface. The function's return does not indicate successful authorization. Check the status by calling the isAuthorized interface.

```
public static boolean authPrepare(Context context);//MegSafe
public static boolean authPrepare(Context context, final String licensefile);//MegSafe
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| context | String | Required | Context of Android Program |
| licensefile | String | Selectable | Customized authorized bin file |

### returned value

Boolean type. True indicates that SDK initialization and authorization initialization are complete.

## getAuth

### summary

Static function.

Get authorization. This requires a one-time call when the device is shipped from the factory to connect to Megvii's server for authorization. The authorization is permanent for the device and does not require reauthorization after successful completion.

Note: When using MegSafe soft authorization solutions, this interface requires no attention. The function's return value does not indicate authorization success. To check authorization status, call the isAuthorized interface. If unsuccessful, retry the getAuth interface. If authorization consistently fails, check the network connection. If authorization is already granted, calling the getAuth interface will not trigger any action.

```
public static void getAuth(final String authURL, final String apiKey, final String apiSecret);//MegSafepublic
static void getAuth(final String authURL, final String apiKey, final String apiSecret, final booleanisTestKey);//
MegSafe
```

parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| authURL | String | Required | The address of the authorized server, such as www.megvii-inc.com or IP address such as 192.168.1.1. |
| apiKey | String | Required | The API key requested by the client is used to verify identity and obtain authorization. |
| apiSecret | String | Required | The API Secret requested by the client is used to verify identity and obtain authorization. |
| isTestKey | boolean | Selectable | If the client's key is a temporary authorization for initial testing, set it to true. |

### returned value

not have

## isAuthorized

### summary

Static function.

Check if the device is authorized. After calling the getAuth API, you must call this API to check if authorization is successful. If not, wait for a while and try again. The getAuth API will keep trying to connect to the authorization server.

You can also get the hardware authorization status by calling this API.

Note: If you use the MegSafe soft authorization solution, you do not need to pay attention to this interface.

```
public static boolean isAuthorized();
```

parameter declaration

not have

### returned value

Boolean type, where true indicates successful authorization.

## getVersion

### summary

Static function.

Get the version number of the current client SDK.

```
public static String getVersion();
```

### parameter declaration

not have

### returned value

A string in the format "1.1.0" indicates the current version number.

## getFeatureVersion

### summary

Get the version number of the current feature.

```
public String getFeatureVersion();
```

**parameter declaration**

not have

### returned value

A string in the format "0" or "112105003" indicates the current feature version number. "0" means the feature model has no valid version information.

## feedFrame

### summary

Transmit a camera frame to the SDK for face detection and quality filtering, then return the detection results.

```
public FacePassDetectionResult feedFrame(FacePassImage image) throws FacePassException;
public FacePassDetectionResult feedFrame(Bitmap BMPImage, int rotation) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| image | FacePassImage | Yes | Convert the camera frame into a FacePassImage object. The type parameter is defined in the FacePassImage class.<br>It mainly includes raw flow,width and height, rotation, encoding, etc. There is no trackId in this scenario.<br>The client must process the camera input frame and initialize the Image object with the specified parameters. See the example for details. |
| BMPImage | Bitmap | Yes | An image in Bitmap format. |
| rotation | int | Yes | The image rotation angle values differ from the forward orientation (face-up direction). Only four valid DEG values (DEG0, DEG90, DEG180, DEG270) defined in FacePassImageRotation represent the rotation angles of 0°,90°,180°, and 270°.<br>This parameter requires user control. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### returned value

FacePassDetectionResult object, see the class definition for details.

feedFrame returns a FacePassDetectionResult object, which mainly includes:

- message/imageList: These two are complementary. The message contains both detection and control information (in encrypted form).
- faceList: This array (in plain text) contains facial detection information (FacePassFace) provided to customers.

The system actually sends the message to the recognition system rather than the face list. If the message is empty, it means there are no faces available for recognition (e.g., faces that failed quality screening).

Call the recognition interface:

Call the recognize method of FacePassHandler, passinggroupName and message to perform facial recognition.

The function returns a FacePassRecognitionResult object, notifying the client of essential information while the handler determines whether to retry or terminate the process.

This also clarifies the relationship between tracking and recognition:

1. The entire recognition process is feedFrame-driven, and upon detecting a valid face, it will prompt the client to initiate a recognition request.

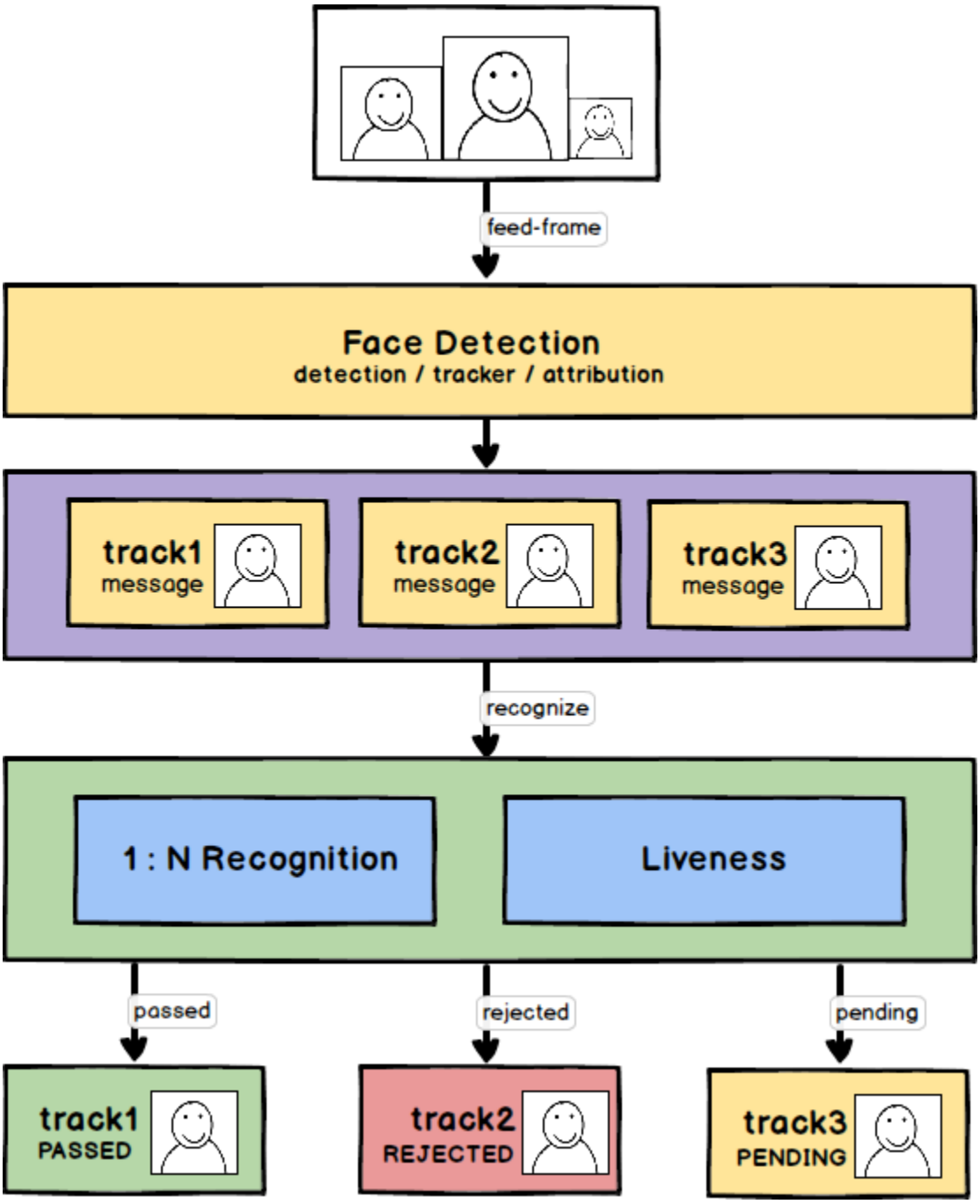
2. The feedFrame function returns multiple faces within a single image frame, which are distinguished by their trackId.

3. The recognition results are displayed by track, showing each track's status rather than individual frames.

4. Different tracks are independent and mutually unaffected.

5. For the same track with different images sent for recognition, the track's status is determined by the results of the returned images (as defined in FacePassRecognitionResult). If the track's status is ending (excluding AWAIT), subsequent recognition results will not affect its status.

Here is a diagram of the entire detection and identification process:




## feedFrameRGBIR

### summary

The RGBIR binocular module transmits a pair of images from the RGB and IR cameras to the SDK for face detection and quality filtering, then returns the detection results of this frame.

```
public FacePassDetectionResult feedFrameRGBIR(FacePassImage imageRGB, FacePassImage imageIR) throws
FacePassException;
public FacePassDetectionResult feedFrameRGBIR(Bitmap BMPImageRGB, int rotationRGB, Bitmap BMPImageIR, int
rotationIR);
```

**Parameter Declaration**

| | Parameter name | Parameter type | Yes or No Required | Parameter declaration |
|---|---|---|---|---|
| feedFrameRGBIR interface 1 | imageRGB | FacePassImage | Yes | Convert RGB camera frames into FacePassImage objects (type parameters defined in FacePassImage class), including raw stream, width/height, rotation, and encoding. Note that trackId is not required in this scenario. The client must process the camera input frames and initialize the Image object with these parameters, as shown in the example. |
| | imageIR | FacePassImage | Yes | Convert IR camera frames into FacePassImage objects (type parameters defined in FacePassImage class), including raw stream, width/height, rotation, and encoding. Note that trackId is not required in this scenario. The client must process the camera input frame and initialize the Image object with these parameters, as shown in the example. |
| feedFrameRGBIR interface 2 | BMPImageRGB | Bitmap | Yes | An image in Bitmap format from an RGB camera. |
| | rotationRGB | int | Yes | Unlike the clockwise rotation angle values for forward-facing (face-up) orientations, RGB images only support four predefined DEG values in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively.<br>This parameter requires user control. If the image is rotated at an incorrect angle, the system may fail to detect the face value. |
| | BMPImageIR | Bitmap | Yes | Bitmap images from IR cameras. |
| | rotationIR | int | Yes | In IR images, the clockwise rotation angle values differ from those in forward-facing (face-up) orientations. Only four valid DEG values (DEG0, DEG90, DEG180, DEG270) defined in FacePassImageRotation represent the rotation angles of 0°,90°,180°, and 270° respectively.<br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### returned value

FacePassDetectionResult object, see the class definition for details.

`feedFrameRGBIRFacePassDetectionResult`

- message/imageList: These two components are complementary. The message contains both detection and control information in encrypted form.
- faceList: This array (in plain text) contains facial detection information (FacePassFace) provided to customers.

The system actually sends the message to the recognition system rather than the face list. If the message is empty, it means there are

no faces available for recognition (e.g., faces that failed quality screening). Call the recognition interface:

Call the recognize method of FacePassHandler, passinggroupName and message to perform facial recognition.

The function returns a FacePassRecognitionResult object, notifying the client of essential information while the handler determines whether to retry or terminate the process.

## recognize

### summary

Based on the FacePassDetectionResult from the feedFrame, perform facial recognition and liveness detection. The results are returned as a FacePassRecognitionResult array, where a single detection may contain multiple faces, with each array entry representing one face.

Face recognition: Identifies the most similar face to the target image among N individuals. If the similarity exceeds the threshold, the recognition is successful; otherwise, it fails. When using the topK recognition interface, it returns the K most similar faces, with each face individually marked as passing or failing the threshold.

The model and threshold are determined by thesearchModel and searchThreshold set during FacePassConfig initialization, or can be customized via parameters for specific trackId. The accuracy of facial recognition depends on factors such as database size, database quality, and on-site lighting conditions.

Liveness detection: Determining whether a face is genuine is independent of the database and primarily depends on the live environment and camera image quality.

The recognize interface is divided into two types: the bottom database recognition interface and the bottomless database recognition interface. For the bottom database interface, you must first create a database and add faces, then pass the database name when calling the recognize function. The bottomless database interface, primarily used in human-document-machine systems, requires no database creation. Instead, it accepts a face feature array (which essentially serves as the database).

**Bottom Library Identification Interface**

```
FacePassRecognitionResult[] recognize(String groupName, byte[] message) throws FacePassException;
FacePassRecognitionResult[] recognize(String groupName, byte[] message, FacePassTrackOptions[] opt) throws
FacePassException;
FacePassRecognitionResult[][] recognize(String groupName, byte[] message, int topK) throws FacePassException;
FacePassRecognitionResult[][] recognize(String groupName, byte[] message, int topK, FacePassTrackOptions[] opt)
throws FacePassException;
```

parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| groupName | String | Required | The group name for face search. |
| message | byte[] | Required | The string for facial recognition is encrypted and generated by the algorithm based on the detection results. It contains some information fields required for facial recognition and serves as the form field for the recognition protocol.<br>The string length is variable and generated by feedFrame. Theoretically, longer strings represent more complex information (e.g., more faces). Therefore, ensure sufficient string length space for external calls.<br>This field is not empty if no face is detected. |
| topK | int | Selectable | The value must be greater than 0 to control the recognition to return multiple similar faces. This interface returns a 2D array, with each dimension containing the topK similar recognition results.<br>If the topK exceeds the bottom database count, the system will return the topK results. However, all results exceeding the bottom database count will have a recognition score of-100.0. |
| opt | FacePassTrackOptions | Selectable | Specify the threshold parameter for this trackId recognition. See FacePassTrackOptions for details. |

**Bottomless database identification interface**

```
FacePassRecognitionResult[][] recognize(FacePassFeature[] features, byte[] message) throws FacePassException;
FacePassRecognitionResult[][] recognize(FacePassFeature[] features, byte[] message, FacePassTrackOptions[] opt)
throws FacePassException;
```

parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| features | FacePassFeature[] | Required | The face feature array, which can be directly obtained from the result returned by the extractFeature interface. For details, refer to the return result of the extractFeature method. |
| message | byte[] | Required | The string for facial recognition is encrypted and generated by the algorithm based on the detection results. It contains some information fields required for facial recognition and serves as the form field for the recognition protocol.<br>The string length is variable and generated by feedFrame. Theoretically, longer strings represent more complex information (e.g., more faces). Therefore, ensure sufficient string length space for external calls.<br>This field is not empty if no face is detected. |
| opt | FacePassTrackOptions | Selectable | Specify the threshold parameter for this trackId recognition. See FacePassTrackOptions for details. |

## returned value

Returns an array of FacePassRecognitionResults, with each item representing the recognition

result of a single image. The member variables are as follows:

1. trackId: trackId, used to link different faces.

2. faceToken: facial ID;

3. Details: specify the classification criteria and thresholds to provide more granular information.

4. Recognition State: The outcome of recognition, serving as the basis for the client's decision-making process, upper-level logic, and GUI.

5. searchErrorCode: error code for search during recognition;

6. livenessErrorCode: Error code indicating liveness during recognition.

For details, see the class definition of FacePassRecognitionResult.

## Supplementary Specification

| | Explain |
| --- | --- |

| | |
|---|---|
| The bottomless database recognition interface returns a two-dimensional array FacePassRecognitionResult[][] as the result. | Since this is a bottomless database comparison, the returned result contains an invalid faceToken value. Other fields indicate the status and details of the feature comparison results. For more information, refer to the FacePassRecognitionResult class.<br><br>Each dimension stores a score derived from comparing a face with a trackid against the FacePassFeature[] input parameters, with the order matching the features array. For example:<br><br>For example, the user extracts features from three images, stores them in a featureArray, and passes this array as input to the recognize interface, as shown below:<br>`result = recognize(featureArray, message);`<br><br>1) When only one face with trackid=1 is detected in the message:<br><br>result[0][0] saved the alignment result between trackid 1 and featureArray[0];<br><br>result[0][1] stores the alignment result between trackid 1 and featureArray[1].<br><br>result[0][2] stores the alignment result between trackid 1 and featureArray[2].<br><br>2) When the message contains two trackids (trackid 8 and trackid 9) for facial recognition<br><br>result[0][0] saved the alignment result between trackid 8 and featureArray[0];<br><br>result[0][1] stores the alignment result between trackid 8 and featureArray[1];<br><br>result[0][2] stores the alignment result between trackid 8 and featureArray[2].<br><br>result[1][0] stores the alignment result between trackid 9 and featureArray[0];<br><br>result[1][1] stores the alignment result between trackid 9 and featureArray[1].<br><br>result[1][2] stores the alignment results between trackid 9 and featureArray[2]. |

## setIRConfig

### summary

Dual-camera image registration parameters. This function configures the positional offset between dual cameras. Configure this once during the SDK lifecycle, before calling ir_filter. The offset coefficient must be determined by the customer based on their device testing.

### parameter declaration

`left_right_k`

`left_right_b`

`top_bottom_k`

`top_bottom_b`

`overlap_rate RGBIR0~1.0IR`

```
public boolean setIRConfig(double left_right_k, double left_right_b, double top_bottom_k, double top_bottom_b,
double overlap_rate) throws FacePassException;
```

## getAgeGender

### summary

To retrieve age and gender, pass the message returned by feedFrame, similar to recognize. This function is only available when the age and gender model is enabled during handler initialization.

```
public FacePassAgeGenderResult[] getAgeGender(byte[] message) throws FacePassException;
```

## setMessage

### summary

Set the track status of the track_id to determine whether the current track_id is sent for recognition.

```
public void setMessage(long trackId, int trackIdState);
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| trackId | long | Yes | A long ID used as a track identifier to mark a person in continuous motion. |
| trackIdState | int | Yes | Refer to the description of the FacePassTrackIdState class. It supports three states, with the following specific mappings:<br>0: "TRACK_ID_RETRY" sets the track status of the current track_id to retry, resetting both the track status and retry attempts for that track_id, and continues generating recognition messages.<br>1: "TRACK_ID_pass" sets the track status of the current track_id to pass, preventing the generation of messages for identification.<br>"2": "TRACK_ID_UNPASS" sets the track status of the current track_id to unpass, and stops sending recognition messages. |

### returned value

not have 。

## createLocalGroup

### summary

Add a new database.

The database serves as a container concept. Our facial recognition system operates through 'one image' and 'one database' matching, rather than exposing a direct 1:1 matching interface for user polling. This approach is due to the algorithm's deep optimization for 1:N retrieval, making it the most cost-effective solution currently available.

Facial registration merely adds a face to the system and extracts feature values (indexed with faceToken). For actual comparison, the faceToken must be linked to a specific group, enabling the client to compare captured images with the corresponding group, thus enabling 1:N facial recognition retrieval.

```
public bool createLocalGroup(String groupName) throws FacePassException;
```

#### The database management API includes:

- Create new database
- Delete the base database
- Query the database
- Face Registration
- Bind face
- Unlink face
- Search for faces in the database
- Download face images

For details, refer to the FacePassHandler API.

### parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| groupName | String | Yes | The newly added localGroupgroupName,which is specified by the user as a parameter, is used for group operations such as adding, deleting, and querying. It also serves as the unique identifier for binding or unbinding faceToken to the group. Parameter restrictions (if not met, an exception of parameter error will be thrown):<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and half-width underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16}<br>The group name can be freely specified. |

**Returned Value**

Boolean type, indicating whether the create operation is completed.

If the group name already exists, the operation fails.

## initLocalGroup

### summary

Loading facial data from the database into memory allows users to load multiple group names. If facial data isn't preloaded, the system will need to retrieve it during the first recognition attempt, causing delays or even timeouts. Therefore, preloading facial data into memory can significantly improve the initial recognition speed.

If the underlying database is large, this interface may take longer to execute. When handling UI interactions, users should consider whether to start a separate thread for this operation and prompt them to disable other database-related operations during execution.

This interface is typically used to initialize pre-existing database face data. For newly created group names, the bindGroup operation (refer to the bindGroup interface documentation) automatically adds face data to memory, eliminating the need for an additional initLocalGroup call.

```
public int initLocalGroup(String groupName) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| groupName | String | Yes | Database name. |

**returned value**

| Parameter type | Parameter declaration |
| --- | --- |
| int | **The number of faces in the corresponding database should be a positive number.**<br>**-1: Initialization failed. The database does not exist.**<br>**-2: Indicates initialization failure or other exceptions;**<br>**0: indicates that the database contains no face data** |

## deleteLocalGroup

### summary

Deleting the base database group will also remove the binding between the group and faceToken.

```
public bool deleteLocalGroups(String groupName) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| groupName | String | Yes | Name of the local group to delete. |

### returned value

Returns true if the group exists, or false if it does not exist or if there is an internal error.

## clearAllGroupsAndFaces

**summary**

Clear the entire database and delete all images.

If the underlying database is large, this interface may take longer to execute. When handling UI interactions, users should consider whether to start a separate thread for this operation and prompt them to disable other database-related operations during execution.

```
public bool clearAllGroupsAndFaces() throws FacePassException;
```

### parameter declaration

not have

### returned value

A boolean return value, where true indicates successful execution and false indicates failure.

## resetDynamicLocalGroup

### summary

Clear all dynamic database data or the dynamic database data of a specific face

```
public void resetDynamicLocalGroup() throws FacePassException;
public void resetDynamicLocalGroup(byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Occurrence | Parameter declaration |
| --- | --- | --- | --- |
| faceToken | byte[ ] | Selectable | The unique facial identifier, known as faceID, allows users to bind or unbind their faceToken to the database, and also supports face deletion and query operations. When no parameters are specified, the interface will clear all dynamic database data.<br>When a specific faceToken is passed, the interface will only clear the dynamic database data for that faceToken. |

### returned value

not have 。

## getLocalGroups

### summary

Query all database groups and list their corresponding group names.

By default, group and face operations are SDK-level operations, while clients implement business-level face and group lists through SDK interfaces.

The group and user operations in SDK are designed to establish the core data structure at the foundational level, enabling efficient algorithmic processing and delivering reliable results to the upper layers. The upper-layer business layer primarily handles practical business logic operations and GUI interactions.

Therefore, at the SDK level, we do not recommend using getLocalGroups directly for GUI operations, nor do we provide features such as cursor navigation or page turning.

```
public String[] getLocalGroups() throws FacePassException;
```

**parameter declaration**

### not have

**Returned Value**

| Parameter name | Parameter type | Occurrence | Parameter declaration |
| --- | --- | --- | --- |

| | | | |
|---|---|---|---|
| groups | Array | If no group is found, return an empty array. | Returns an array of groups, each element being a group name string, with the following rules:<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16}<br>If no group exists in the system, return an empty array (no NULL value is returned, and the groups field is not omitted). |

Returns a local group'sgroupName list from the String array, excluding internal information such as id, version, and facetoken.

## getLocalGroupInfo

### summary

Use the parametergroupName to query group details.

```
public byte[][] getLocalGroupInfo(String groupName) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| groupName | Required | String | The group name, specified by the user as a parameter, is used for group operations (adding, deleting, or querying) and serves as the unique identifier for binding or unbinding faceToken. Parameter restrictions (if not met, a general parameter error code is returned):<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16} |

**Return parameter**

| Parameter name | Parameter type | Occurrence | Parameter declaration |
|---|---|---|---|
| faceToken | byte[][] | Must exist | A string array with each string containing 24 to 100 visible characters, generated by the system after successful database entry. This unique identifier for faces, known as faceID, allows users to The faceToken can be used to bind/unbind the database, as well as to perform face deletion and query operations.<br>Returns an array of faceToken results. |

## getLocalGroupFaceNum

### summary

Query the number of faces in the group using the parametergroupName.

```
public int getLocalGroupFaceNum(String groupName) throws FacePassException;
```

**Parameter Declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| groupName | Required | String | The group name, specified by the user as a parameter, is used for group operations (adding, deleting, or querying) and serves as the unique identifier for binding or unbinding faceToken. Parameter restrictions (if not met, a general parameter error code is returned):<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16} |

### Return parameters

Number of faces, an integer, where negative values indicate query failure.

## addFace

### summary

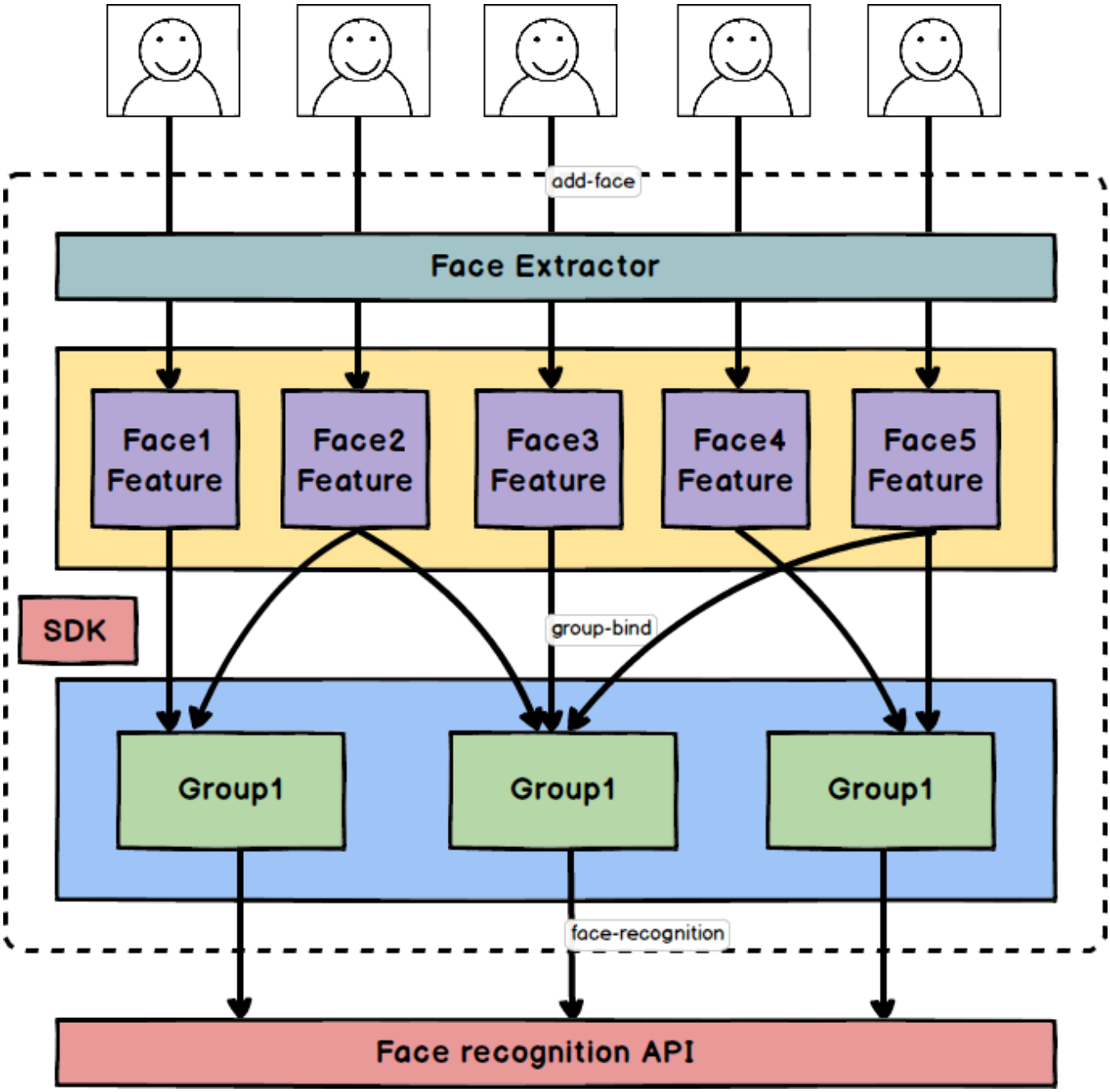
The following section will explain the logical relationship between addFace and the underlying database.

The database is a collection of facial feature values containing the faces that customers want to be 'identified'.

FacePass's facial recognition relies on a database, with each facial recognition session requiring a target image and a database name as input parameters.

Face registration is performed via the addFace API. The SDK employs an extractor to reduce a facial image's dimensionality into feature values for recognition algorithms. The facial feature database operates independently of the underlying database, while the face database exists as a pool. The underlying database and the face database maintain a binding relationship.

In other words, once facial registration is completed, all data enters a centralized facial database pool. Each database can then select target individuals from this pool for binding.




The facial recognition process works as follows: It takes a byte stream of an image, detects the largest face within it, and performs a quality assessment. If all steps are correct, the system outputs the corresponding faceToken; otherwise, it throws an error.

1. The input format is Bitmap. We recommend limiting the resolution size. While oversized images can be used, they may cause slow processing.

2. You cannot add a new item if the image does not contain a face.

3. The system requires a minimum face size of 100×100 pixels for detection, which is the minimum size required for Bitmap recognition.

4. If the image contains multiple faces, detect the largest face (provided the minimum face size requirement is met).

5. If the face detection passes, the system performs feature extraction to degrade the image into feature values (the data structure used in actual face recognition comparisons) and stores them, then indexes them through faceToken.

6. When successful, return FaceToken, result, and the Rect face box position.

7. Returns the corresponding error code if the operation fails.

8. Registering a face and linking it to the database are two separate processes. First, a face must be added to the system and assigned a faceToken, which can then be linked to multiple groups.

9. Adding faces is a time-consuming operation. We recommend calling this method in a child thread. If you call this method in the main thread, avoid adding large images to prevent ANR.

```
public FacePassAddFaceResult addFace(FacePassImage image) throws FacePassException;
public FacePassAddFaceResult addFace(Bitmap BMPImage) throws FacePassException;
public FacePassAddFaceResult addFace(Bitmap BMPImage, int rotation) throws FacePassException;
```

**Parameter Declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| image | Yes | FacePassImage | Convert the input image into a FacePassImage object. The type parameter is defined in the FacePassImage class. |
| BMPImage | Yes | Bitmap | **Image property restrictions:**<br>1 Bitmap resolution limit:<br>• Recommended resolution: 1280×720<br>• Minimum supported face size: Must exceed FaceMin. The addFace interface provides a dedicated threshold setting (see setAddFaceConfig).<br>No maximum size limit is currently available. Avoid oversized images; recommended dimensions: 2560*2560 or smaller.<br>• Resolution constraints apply to 'area', so oddly proportioned images may pass if the faces meet the minimum pixel requirements.<br>Exceeding these limits will trigger the FacePassException 'InvalidImageResolution'.<br>2. Image rotation angle:<br>• Some images are rotated. You need to parse the EXIF information and set the rotation parameter.<br>3. other：<br>• The primary goal of background selection is to ensure clarity and consistency with the user.<br>• Black and white images are not recommended<br>• We do not recommend using strong Photoshop effects or artistic images<br>• It is recommended to use recent photos of your daily life. Avoid using photos that are too old.<br>• Make sure your makeup and glasses are consistent with your daily habits<br>**Face attribute restrictions:**<br>The integrated detection process is:<br>• Detect faces in images;<br>• Locate the largest face and perform quality assessment (including FaceMin);<br>• If the quality is judged to be successful, the feature value is extracted.<br>• Exit if any step fails.<br>The following restrictions and notes apply:<br>1. Number of faces in the same frame:<br>• The database import process supports only single faces, regardless of the number of faces in the original image.<br>• If there are multiple faces, select only the largest one.<br>• If the largest face fails, it will **not look at the second largest face** and will fail directly, so make sure the largest face in the original image is the user's target face.<br>2. Inventory quality judgment (SDK default): |

| Broad heading | Subclass | numeric value | Remarks |
|---|---|---|---|
| Pose | Yaw | <= 20° | Horizontal angulation |
| | Pitch | <= 20° | Vertical angle |
| | Roll | <= 20° | Angle of rotation |
| Blur | Blurriness | <= 0.7 | Articulation |
| Illumination | Brightness | [65, 210] | Average face brightness |
| | Std_Deviation | <= 80 | Standard deviation of face brightness |
| FaceMin | FaceMin | >= 100 | Minimum face |

- The addFace interface includes a dedicated threshold configuration feature (see setAddFaceConfig). When certain images must be stored in the database but fail to meet the default threshold, users can adjust the threshold manually via setAddFaceConfig.
- Minimum face size (FaceMin or min-face):

  Facial recognition requires a minimum face size in the comparison images, as smaller faces would significantly reduce accuracy and impair recognition performance.

  - The default minimum face size is: length ≥ 100px, width ≥ 100px. Note: The equal sign is valid, meaning a 100x100 image can be used.
  - Theoretically, there's no maximum face size limit. However, oversized faces may slightly delay the feature extraction process compared to normal conditions, but this is a one-time operation and is generally acceptable by default.
  - If two faces are of identical size, the algorithm will select the first one detected.
- Angle pose: Large angles may cause facial information loss. We impose restrictions across three axes, with boundary violations triggering error alerts.
  - roll <= 20°: rotation of the face plane around the nose as the axis;
  - yaw <= 20°: head movement left and right;
  - Pitch <= 20°: Nodding up and down;
- Blur: Blurred photos can degrade recognition accuracy. Ensure images are clear during the database upload phase.

| | | | |
|---|---|---|---|
| | | | • Motion blur: The trailing effect caused by high-speed movement in images.<br>• Gaussian blur: occurs when a camera's out-of-focus image causes pixel blending with surrounding areas.<br>• Other blurring: such as photo retouching and other factors;<br>• Illumination: Facial recognition is still quite sensitive to lighting conditions. We recommend using soft, well-lit environments with no harsh contrast.<br>• The current version lacks specific optimization for backlighting and back lighting.<br>• This version does not have special optimization for low light.<br>• We recommend an average brightness of [65-210] lux with a standard deviation of <= 80 lux. |
| rotation | Selectable | int | The image rotation angle relative to the forward orientation (face-up direction) is limited to four valid values defined in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively. The default value is DEG0.<br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### Return parameters

See the class definition of FacePassAddFaceResult for details.

## deleteFace

### summary

Delete face.

1. Delete the corresponding faceToken;

2. Remove the bound faceToken relationship from the group.

```
public boolean deleteFace(byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required | Parameter type | Parameter declaration |
|---|---|---|---|
| faceToken | Required | byte[ ] | A visible string with length 24-100 characters, generated by the system after successful database entry (see addFace). This unique identifier for faces is referred to as faceID.<br>The encoding rules for faceToken are:<br>• Length 24 to 100<br>• Supports lowercase letters, uppercase letters, and half-width English equal signs, underscores, and strikethroughs<br>However, in actual API processing, the system does not strictly validate encoding formats but directly decrypts the data into plaintext. If decryption fails, it throws a FacePassException. Users can bind or unbind the underlying database using faceToken, and it can also be used for deleting or querying face data.<br>The faceToken is linked to multiple groups. To delete it, you must first unlink it from all groups before proceeding with the deletion.<br>For binding faceToken and group, refer to:<br>FacePassHandler's bindGroup/unbindGroup |

### Return parameters

A boolean value, true if deleted successfully, false otherwise.

## extractFeature

### summary

Feature extraction extracts facial feature values from images. For images with multiple faces, only the largest face's feature values are extracted. This interface requires exception capture.

```
public FacePassExtractFeatureResult extractFeature(FacePassImage image) throws FacePassException;
public FacePassExtractFeatureResult extractFeature(Bitmap BMPImage) throws FacePassException;
public FacePassExtractFeatureResult extractFeature(Bitmap BMPImage, int rotation) throws FacePassException;
```

### Parameter Description

| | | | |
|---|---|---|---|

<table>
<tr><th>Parameter name</th><th>Is it required?</th><th>Parameter type</th><th>Parameter declaration</th></tr>
<tr><td>image</td><td>Yes</td><td>FacePassImage</td><td>Convert the input image into a FacePassImage object. The type parameter is defined in the FacePassImage class.</td></tr>
<tr><td>BMPImage</td><td>Yes</td><td>Bitmap</td><td>
Image property restrictions:<br><br>
1 Bitmap resolution limit:<br>
• Recommended resolution: 1280×720<br>
• Minimum face size: Must exceed FaceMin. The extractFeature interface includes a dedicated threshold setting (see setAddFaceConfig). No maximum size limit is currently available. Avoid oversized images; the recommended dimensions are 2560*2560 or smaller.<br>
• Resolution constraints apply to 'area', so oddly, images with unusual aspect ratios may pass, provided their faces meet the minimum pixel size requirement.<br>
• If the boundary is exceeded, it throws a FacePassException with the message "Invalid Image Resolution".<br><br>
2. Image rotation angle:<br>
• Some images are rotated. You need to parse the EXIF information and set the rotation parameter.<br><br>
3. other：<br>
• The primary goal of background selection is to ensure clarity and consistency with the user.<br>
• Black and white images are not recommended<br>
• We do not recommend using strong Photoshop effects or artistic images<br>
• It is recommended to use recent photos of your daily life. Avoid using photos that are too old.<br>
• Make your makeup and glasses as consistent as possible with your daily habits<br><br>
Face attribute restrictions:<br><br>
The integrated detection process is:<br>
• Detect faces in images;<br>
• Locate the largest face and perform quality assessment (including FaceMin);<br>
• The feature value is extracted by the quality judgment.<br>
• Exit if any step fails.<br><br>
The following restrictions and notes apply:<br><br>
1. Number of faces in the same frame:<br>
• The feature extraction process supports only single faces, regardless of the number of faces in the original image.<br>
• If there are multiple faces, select only the largest one.<br>
• If the largest face fails, it will not look at the second largest face and will fail directly, so make sure the largest face in the original image is the user's target face.<br><br>
2. Inventory quality judgment (SDK default):<br><br>
<table>
<tr><th>Broad heading</th><th>Subclass</th><th>numeric value</th><th>Remarks</th></tr>
<tr><td rowspan="3">Pose</td><td>Yaw</td><td><= 20°</td><td>Horizontal angulation</td></tr>
<tr><td>Pitch</td><td><= 20°</td><td>Vertical angle</td></tr>
<tr><td>Roll</td><td><= 20°</td><td>Angle of rotation</td></tr>
<tr><td>Blur</td><td>Blurriness</td><td><= 0.7</td><td>Articulation</td></tr>
<tr><td rowspan="2">Illumination</td><td>Brightness</td><td>[65, 210]</td><td>Average face brightness</td></tr>
<tr><td>Std_Deviation</td><td><= 80</td><td>Standard deviation of face brightness</td></tr>
<tr><td>FaceMin</td><td>FaceMin</td><td>>= 100</td><td>Minimum face</td></tr>
</table>
<br>
• The extractFeature interface includes a dedicated threshold configuration feature (see setAddFaceConfig). When certain images require feature extraction but fail to meet the default threshold, users You can change the threshold yourself through setAddFaceConfig. Minimum face size (FaceMin or min-face):<br><br>
Facial recognition requires a minimum face size in the comparison images, as smaller faces would significantly reduce accuracy and impair recognition performance.<br>
• The default minimum face size is: length ≥ 100px, width ≥ 100px. Note: The equality is valid, meaning a 100x100 image can be used.<br>
• Theoretically, there's no maximum face size limit. However, oversized faces may slightly slow down the feature extraction process compared to normal conditions, but this is a one-time operation and is generally acceptable by default.<br>
• If two faces are of identical size, the algorithm will prioritize the first one detected.<br>
• Angle pose: Large angles may cause facial information loss. We impose three-axis constraints with boundary violation triggering error. Roll <= 20°: Nose-axis rotation of the facial plane.<br>
• yaw <= 20°: head movement left and right;<br>
• Pitch <= 20°: Nodding up and down;<br>
• Blur: Blurred photos may affect recognition accuracy. We recommend keeping images sharp. Motion blur: High-speed movement causes trailing effects in images.<br>
• Gaussian blur: occurs when a camera's out-of-focus image causes pixel blending with surrounding areas.<br>
• Other blurring: such as photo retouching and other factors;<br>
• Illumination: Facial recognition is still quite sensitive to lighting conditions. We recommend using soft, well-lit environments with no harsh contrast.<br>
• This version does not have special optimization for backlighting. This version does not have special optimization for low light.<br>
• We recommend an average brightness of [65-210] lux with a standard deviation of <= 80 lux.
</td></tr>
<tr><td>rotation</td><td>Selectable</td><td>int</td><td>The image rotation angle relative to the forward orientation (face-up direction) is limited to four valid values defined in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively. The default value is DEG0.</td></tr>
</table>

| | | | |
|---|---|---|---|
| | | | This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### Return parameters

See the class definition of FacePassExtractFeatureResult for details. The result is unavailable if the interface throws an exception.

## insertFeature

### summary

Feature storage is used to add feature values extracted from other devices to the local face database. Exceptions must be captured when using this interface.

```
public String insertFeature(byte[] featureData, FacePassFeatureAppendInfo featAppendInfo) throws
FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
|---|---|---|---|
| featureData | Required | byte[ ] | Feature data typically measures 256,512,1024, or 2048 bytes (varies by feature model, as per actual specifications).<br>Users must ensure their feature data model version matches the feature model version deployed on the device. |
| featAppendInfo | Selectable | FacePassFeatureAppendInfo | Optional supplementary information for feature storage, as defined in FacePassFeatureAppendInfo. |

### Return parameters

A faceToken of type String.

- If the user-configured parameter featAppendInfo faceToken is valid, the returned faceToken matches the user's specified value.
- If the user-configured parameter featAppendInfo faceToken is invalid, the system returns a 24-byte faceToken string randomly generated by the SDK.
- If the user's featureData is empty or not 256,512, or 2048 bytes, an exception is thrown. For other internal SDK exceptions, the returned faceToken is null and unavailable.

## getFaceImage

### summary

Download the image content.

Currently, the SDK only stores basic database faces, with each faceToken corresponding to a single image.

Note that the image stored here is not the original image from the database, but a cropped version extracted based on facial positioning. It is typically used for GUI display. To retrieve the original image, pass the faceToken, which will return the cropped facial image. The current cropping rule is:

- The cropped image is twice the height and width of the original rectangle.
- Expand the width while keeping the center point unchanged;
- Expand the height by 3/5 at the top and 2/5 at the bottom (this translates to a 1/10 upward shift of the center point).

```
public Bitmap getFaceImage(byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required | Parameter type | Parameter declaration |
|---|---|---|---|
| faceToken | Required | byte[ ] | Specify faceToken to return the corresponding face mask.<br>A visible string of 24 to 100 characters, generated by the system after successful face registration (see addFace), serving as the unique identifier for the face. |

### Return parameter

Bitmap, used to display operations.

## getFaceImagePath

### summary

Currently, the SDK only stores basic database faces, with each faceToken corresponding to a single image.

The storage contains small images extracted from face positions, typically used for GUI display. The query method takes a faceToken as input and returns the path of the corresponding image stored during storage.

```
public String getFaceImagePath(byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
| --- | --- | --- | --- |
| faceToken | Required | byte[ ] | Specify faceToken to return the corresponding face mask.<br>A visible string of 24 to 100 characters, generated by the system after successful face registration (see addFace), serving as the unique identifier for the face. |

### Return parameters

String, faceToken: the storage path for the corresponding image.

## getFeatureByFaceToken

### summary

Get the facial features corresponding to the faceToken.

The query method takes faceToken as input and returns the corresponding facial features.

```
public byte[] getFeatureByFaceToken(byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
| --- | --- | --- | --- |
| faceToken | Required | byte[ ] | Specify faceToken to return the corresponding face mask.<br>A visible string of 24 to 100 characters, generated by the system after successful face registration (see addFace), serving as the unique identifier for the face. |

### Return parameter

byte[], where faceToken corresponds to facial features.

## bindGroup

### summary

Bind a face to the database for recognition.

Here we summarize the key APIs from previous versions and explain their core logic:

1. "Face registration" primarily serves "face access control", ensuring an image meets quality standards for retrieval. However, the face cannot be directly compared, as all searches are based on group names.

2. All searches are performed usinggroupName, as the algorithm's underlying layer has implemented multiple optimizations (primarily in parallel computing). Thus, users are not exposed to direct faceToken operations, but rather the algorithm's black-box comparison process handles the matching.

3. The faceToken serves as the unique identifier for faces, while the database usesgroupName as its unique identifier.

4. To enable the comparison, bind the faceToken to the group name and configure the target as the group name on the client.

5. The system does not report an error for duplicate binding. Simply rebind as usual.

```
public boolean bindGroup(String groupName, byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
| --- | --- | --- | --- |
| groupName | Required | String | The group name, specified by the user as a parameter, is used for group operations such as adding, deleting, and querying, and serves as the unique identifier for binding or unbinding faceToken. Parameter restrictions (if not met, a general parameter error code is returned):<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16} |
| faceToken | Required | byte[ ] | A visible string of 24 to 100 characters, generated by the system after successful database entry (see addFace), serves as the unique identifier for a face, commonly referred to as a faceID. Users can bind or unbind their faceToken to the database, and it can also be used for face deletion and query operations. |

### Return parameters

A boolean value, where true indicates successful binding and false indicates otherwise.

### Error code

not have

## unbindGroup

### summary

This is the inverse operation of bindGroup, which unlinks a face from the database. More precisely, it disconnects the binding between faceToken andgroupName.

After unbinding, the search for the target group name will no longer include this faceToken. However, since access control systems currently support configuring multiple groups, other group names will continue to be recognized (as they remain unbound). Therefore, if you want to prevent the access control system from recognizing a specific face, make sure all configured groups on the system have been unbound.

If the corresponding binding does not exist, an error will be reported and the unbinding will fail.

```
public boolean unbindGroup(String groupName, byte[] faceToken) throws FacePassException;
```

**parameter declaration**

| Parameter name | Is it required? | Parameter type | Parameter declaration |
| --- | --- | --- | --- |
| groupName | Required | String | The group name, specified by the user as a parameter, is used for group operations (adding, deleting, or querying) and serves as the unique identifier for binding or unbinding faceToken. Parameter restrictions (if not met, a general parameter error code is returned).<br>• Supports 4-16 bit length<br>• Supports lowercase and uppercase letters, numbers, and underscores<br>• Recommended regular expression: /^[a-zA-Z0-9_-]{4,16} |
| faceToken | Required | byte[ ] | A visible string of 24 to 100 characters, generated by the system after successful database entry (see addFace), serves as the unique identifier for a face, commonly referred to as a faceID. Users can bind or unbind their faceToken to the database, and it can also be used for face deletion and query operations. |

### Return parameters

A boolean value, true if unbound successfully, false otherwise.

## addFaceDetect

### summary

The system transmits a camera frame to the SDK for facial detection and recognition, then synchronously returns the detection result. If the facial quality meets the database entry requirements (as determined by the addFaceConfig parameter), the system returns the image to the client for facial registration.

This interface operates similarly to feedFrame. We recommend providing consecutive camera frames to the client, enabling the SDK to effectively utilize its tracking and quality assessment capabilities.

But unlike feedFrame:

- The return value is the result of each current frame, while the feedFrame returns the result of the entire track process.
- Only the largest face information is returned, not the multi-person list, because the face registration logic can only target one face.

Do not use them interchangeably in practice.

```
public FacePassAddFaceDetectionResult addFaceDetect(FacePassImage image) throws FacePassException;
public FacePassAddFaceDetectionResult addFaceDetect(Bitmap BMPImage, int rotation) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| image | FacePassImage | Yes | Convert the camera frame into a FacePassImage object. The type parameter is defined in the FacePassImage class. |
| BMPImage | Bitmap | Yes | An image in Bitmap format. |
| rotation | int | Yes | The image rotation angle values differ from the forward orientation (face-up direction). Only four valid DEG values (DEG0, DEG90, DEG180, DEG270) defined in FacePassImageRotation represent the rotation angles of 0°,90°,180°, and 270°.<br><br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### returned value

The FacePassAddFaceDetectionResult object, as defined in the class.

## detectFaces

### summary

This is a pure image detection interface. Input an image for face detection and return the detection result of this frame simultaneously. No quality filtering is performed.

```
public FacePassDetectFacesResult detectFaces(FacePassImage image) throws FacePassException;
public FacePassDetectFacesResult detectFaces(Bitmap BMPImage, int rotation) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| image | FacePassImage | Yes | The input image is processed into a FacePassImage object, with type parameters defined in the FacePassImage class. |
| BMPImage | Bitmap | Yes | An image in Bitmap format. |
| rotation | int | Yes | The image rotation angle relative to the forward orientation (face-up direction) is limited to four valid values defined in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively. The default value is DEG0.<br><br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |

### returned value

The FacePassDetectFacesResult object, as defined in the class.

## compare

## summary

Import two images for 1:1 comparison.

The two provided images will undergo face detection. If the liveness detection parameter is enabled, liveness detection will also be performed.

If face detection fails, an error will be reported (comparing non-compliant photos is meaningless, and we cannot guarantee the results' practical value).

If all the above steps are correct, a compre score will be returned to determine whether the test passed.

The score ranges from 0 to 100, where higher values indicate greater similarity (the same person) and lower values indicate less similarity (two different people).

The standard practice is to compare the score with a threshold, with a commonly recommended threshold of 70.

Note that unlike standard image usage rules, the compare API currently lacks strict quality checks. This is because it serves as a low-level API, and clients often deploy it in complex scenarios where establishing a robust quality assurance system is challenging.

Therefore, our strategy is to provide detailed parameters in the API return values. Users can then customize their quality system requirements (while disregarding our comparison scores).

```
public FacePassCompareResult compare(FacePassImage image1, FacePassImage image2, boolean enableLiveness) throws
FacePassException;
public FacePassCompareResult compare(Bitmap BMPImage1, Bitmap BMPImage2, boolean enableLiveness) throws
FacePassException;
public FacePassCompareResult compare(Bitmap BMPImage1, int rotation1, Bitmap BMPImage2, int rotation2, boolean
enableLiveness) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Yes or No Required | Parameter declaration |
|---|---|---|---|
| image1 | FacePassImage | Yes | The input image is processed into a FacePassImage object, with type parameters defined in the FacePassImage class. |
| image2 | FacePassImage | Yes | |
| BMPImage1 | Bitmap | Yes | Two images for comparison, in Bitmap format.<br>**Bitmap property restrictions:**<br>1. Resolution limit:<br>• Recommended resolution: 1280×720<br>• Minimum supported face size: Must be larger than FaceMin, so must be at least 100*100.<br>• No size restrictions are currently in place. We recommend keeping images under 2560*2560 pixels to avoid registration delays.<br>Since resolution determines 'area', oddly proportioned images may still pass if the face meets the minimum pixel requirement.<br>• If the boundary is exceeded, it throws a FacePassException with the message "Invalid Image Resolution".<br>2. Image rotation angle:<br>• Some images are rotated. You need to parse the EXIF information and set the rotation parameter.<br>3. other：<br>• The primary goal of background selection is to ensure clarity and consistency with the user.<br>• Black and white images are not recommended<br>• We do not recommend using strong Photoshop effects or artistic images<br>• It is recommended to use recent photos of your daily life. Avoid using photos that are too old.<br>• Make sure your makeup and glasses are consistent with your daily habits<br>**Face attribute restrictions:**<br>In 1:1 scenarios, image input sources are complex, so quality assessment is not performed in this case.<br>The return value provides detailed quality metrics (3D-Pose and Blurness). If necessary, clients may make their own judgments. |
| BMPImage2 | Bitmap | Yes | |
| rotation1 | int | Selectable | The image rotation angle relative to the forward orientation (face-up direction) is limited to four valid values defined in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively. The default value is DEG0.<br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |
| rotation2 | int | Selectable | |
| enableLiveness | boolean | Yes | Check if the object is alive.<br>Note: Since live detection consumes client resources, it is recommended to set it to false for high-frequency use.<br>Additionally, if the Bitmap fails to detect a face, it will immediately throw an error and stop wasting resources on liveness detection. |

**Returned Value**

Return the FacePassCompareResult.

## compare1xN

### summary

This API compares a single face in an image against a database and returns the topK matching results. If the image contains multiple faces, the user must manually filter and select one face, or repeatedly call the API to process all faces.

```
public FacePassSearchResult[] compare1xN(FacePassImage image, byte[] lData, String groupName, int topK) throws FacePassException;
public FacePassSearchResult[] compare1xN(Bitmap BMPImage, int rotation, byte[] lData, String groupName, int topK) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| image | FacePassImage | Yes | The input image is processed into a FacePassImage object, with type parameters defined in the FacePassImage class. |
| BMPImage | Bitmap | Yes | An image in Bitmap format. |
| rotation | int | Yes | The image rotation angle values differ from the forward orientation (face-up direction). Only four valid DEG values (DEG0, DEG90, DEG180, DEG270) defined in FacePassImageRotation represent the rotation angles of 0°,90°,180°, and 270°.<br>This parameter is user-controlled. If the image is rotated at an incorrect angle, the system may fail to detect the face. |
| lData | byte[] | Yes | lData is the return value from the detectFaces interface. |
| groupName | String | Yes | Database name. |
| topK | int | Yes | The topK value must be greater than 0 to return the topK matching results. If topK exceeds the database size, the matching score for the excess part is-100.0. |

### returned value

Return the FacePassSearchResult.

## search

### summary

Compare one feature with the database and return the topK matching results. If there are multiple features, the user must call this interface repeatedly to process all features.

```
public FacePassSearchResult[] search(byte[] featureData, String groupName, int topK) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
|---|---|---|---|
| featureData | byte[] | Yes | Feature data typically measures 256,512,1024, or 2048 bytes (varies by feature model, as specified in the actual implementation). Users must ensure their featureData model version matches the feature model version deployed on the device. |
| groupName | String | Yes | Database name. |
| topK | int | Yes | The topK value must be greater than 0 to return the topK matching results. If topK exceeds the database size, the matching score for the excess part is-100.0. |

### returned value

Return the FacePassSearchResult.

## getConfig

### summary

Retrieve the threshold and switch class configurations from the current FacePassHandler config

```
public FacePassConfig getConfig() throw FacePassException;
```

Parameter description not provided

### returned value

FacePassConfig instance.

## getAddFaceConfig

### summary

Get the current warehouse configuration

```
public FacePassConfig getAddFaceConfig() throw FacePassException;
```

Parameter description not provided

### returned value

FacePassConfig instance.

## setConfig

### summary

Set the config of FacePassHandler. This affects the interface feedFrame.

A typical scenario is:

- Get the current config from Config.
- Modify a public member variable in config;
- Set the new config with setConfig.

Note that config takes effect per feedFrame cycle. Specifically, setConfig becomes active only in the next feedFrame, while previously transmitted protocols remain unaffected.

`setConfig` can only adjust thresholds and enable/disable switches, but cannot modify the model again. The model is finalized during the initial initialization of `FacePassHandler`.

```
public int setConfig(FacePassConfig config) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| config | FacePassConfig | Yes | See the parameter description for the FacePassConfig class |

### returned value

Boolean type: Whether the configuration update is complete

### Example

```
FacePassConfig faceConfig= mFacePassHandler.getConfig();faceConfig.bl-
urThreshold = 0.8f;
```

```
faceConfig. faceMinThreshold = 100;mFacePassHa-
ndler.setConfig(faceConfig);
```

## setAddFaceConfig

### summary

Set the database parameters. Affects the following interfaces: addFace/extractFeature/compare

A typical scenario is:

- Get the current config using getAddFaceConfig.
- Modify a public member variable in config;
- Apply the new config with setAddFaceConfig.

Generally, the parameters used for database storage differ from those in setConfig, and the storage requirements are stricter. By default, after initializing FacePassHandler, the config settings for both database storage and feedFrame interfaces are identical.

```
public boolean setAddFaceConfig(FacePassConfig config) throws FacePassException;
```

**parameter declaration**

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| config | FacePassConfig | Yes | See the parameter description for the FacePassConfig class<br>The current setAddFaceConfig interface supports the following threshold and switch parameters:<br>config.blurThreshold<br>config.poseThreshold<br>config.faceMinThreshold<br>config.lowBrightnessThreshold<br>config.highBrightnessThreshold<br>config.brightnessSTDThreshold<br>config.brightnessSTDThresholdLow<br>config.landmarkThreshold<br>config.lmkoccThreshold<br>config.edgefacecompThreshold<br>config.rcAttributeAndOcclusionMode (This configuration in the repository is valid only for mode 0 and 2. Other values are equivalent to 0) |

### returned value

Boolean type: Whether the configuration update is complete

### Example

```
FacePassConfig addFaceConfig= mFacePassHandler.getAddFaceConfig();
addFaceConfig.blurThreshold = 0.7f;
addFaceConfig.faceMinThreshold = 100;
mFacePassHandler.setAddFaceConfig(addFaceConfig);
```

## reset

### summary

Reset the current handler to its initial state.

The reset function resets track and frame data, representing the system's state after handler initialization. Calling this function without stopping the feedFrame or recognize interface may cause recognition failure.

Use this interface with caution. We recommend discussing the relevant business logic with technical support before using it.

```
public void reset() throws FacePassException;
```

#### parameter declaration

not have

### returned value

not have

## setAffinity

### summary

Static function.

The FacePassHandler member function binds the current thread to the device by setting the mask.

```
public static void setAffinity(int mask);
```

#### parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| mask | int | Required | Device mask meaning:<br>① RK3288:<br>CPU0: 0x01<br>CPU1: 0x02<br>CPU2: 0x04<br>CPU3: 0x08<br>② RK3399:<br>CPU0: 0x01<br>CPU1: 0x02<br>CPU2: 0x04<br>CPU3: 0x08<br>CPU4: 0x10<br>CPU5: 0x20 |

### returned value

not have

## extractFeatureWithLData

### summary

A dedicated feature extraction interface. For a given image frame, the system performs facial detection through addFaceDetect/detectFaces. The function extractFeatureWithLData then extracts corresponding facial feature values from the detected image and returns them. This interface requires exception handling during implementation.

```
public byte[] extractFeatureWithLData(FacePassImage image, byte[] lData) throws FacePassException;
public byte[] extractFeatureWithLData(Bitmap BMPImage, byte[] lData) throws FacePassException;
public byte[] extractFeatureWithLData(Bitmap BMPImage, int rotation, byte[] lData) throws FacePassException;
```

parameter declaration

| Parameter name | Is it required? | Parameter type | Parameter declaration |
| --- | --- | --- | --- |
| image | Yes | FacePassImage | Process the image input to generate lData data and convert it into a FacePassImage object. The type parameter is defined in the FacePassImage class. |
| BMPImage | Yes | Bitmap | The image must correspond to the frame of the generated lData. |
| rotation | Selectable | int | The image rotation angle, relative to the forward orientation (face-up direction), is limited to four valid values defined in FacePassImageRotation: DEG0, DEG90, DEG180, and DEG270, representing 0°,90°,180°, and 270° respectively. The default value is DEG0.<br>This parameter requires user control. If the image is rotated at an incorrect angle, the face value may not be detected. |
| lData | Yes | byte[] | Supported lData data:<br>(1) The lData member in FacePassAddFaceDetectionResult returned by the addFaceDetect interface.<br>(2) The FacePassDetectFace.lData member in the FacePassDetectFacesResult returned by the detectFaces interface. |

### Return parameters

The number of eigenvalues is usually 256,512,1024, or 2048 bytes. Different recognition models may vary, so refer to the specific model's output.

## livenessClassify

### summary

Based on the FacePassDetectionResult from the feedFrame, the system performs liveness detection, returning the results as a FacePassLivenessResult array. A single detection may contain multiple faces, with each array entry representing one. The determination of a face's authenticity is independent of the database and depends on factors such as the live environment and camera image quality.

This interface requires enabling monocular or binocular liveness detection; otherwise, an exception will be thrown.

```
FacePassLivenessResult[] livenessClassify(byte[] message) throws FacePassException;
FacePassLivenessResult[] livenessClassify(byte[] message, FacePassTrackOptions[] opt) throws FacePassException
```

parameter declaration

| Parameter name | Parameter type | Is it required? | Parameter declaration |
| --- | --- | --- | --- |
| message | byte[] | Required | A string for liveness recognition, generated by the algorithm based on the detection results, contains some information fields required for liveness recognition and is used as a form field in the recognition protocol.<br>The string length is variable and generated by feedFrame. Theoretically, longer strings represent more complex information (e.g., more faces). Therefore, ensure sufficient string length space for external calls.<br>This field is not empty if no face is detected. |
| opt | FacePassTrackOptions | Selectable | Specify the threshold parameter for the current live track. See FacePassTrackOptions for details. |

### returned value

Returns an array of FacePassLivenessResults, with each entry representing the recognition

result of a single image. For details, see the FacePassLivenessResult class definition.

The specific logic of feedFrame->livenessClassify is similar to that of feedFrame->recognize.

## fsyncLocalGroups

### summary

Write the database log back to the database. Generally, there is no need to call this interface. When the release algorithm handler is normally executed, this operation will be automatically performed. If the user needs to back up the facepass.db during the interval between using the algorithm, it is best to call this interface first and then back up the facepass.db.

```
public void fsyncLocalGroups();
```

#### parameter declaration

not have

#### returned value

not have

## release

### summary

Release the current SDK handler. This must be done before exiting the app

to prevent memory leaks. Ensure the following calls are performed in pairs:

`mFacePassHandler = new FacePassHandler(config);`

`mFacePassHanlder.release();`

```
public void release() throws FacePassException;
```

#### parameter declaration

not have

#### returned value

not have